



Lecture 12: Post-training (cont.) & Prompting

Instructor: Xiang Ren
USC CSCI 444 NLP
2026 Spring

Logistics

Paper Review & Presentation (starting March 25)

12% of the grade

Each team will complete **one in-depth research paper presentation (10-12 mins presentation + 3-5 mins discussion)** during the semester using slides.

Each student will also serve as a **designated discussion lead for two additional paper presentations.**

Paper Presentation Guidelines

Explain main ideas in an understandable way + discussion question

Present 1 paper (or up to 3 closely related papers) from top NLP venues

- *CL, EMNLP, COLING, COLM, CoNLL

Paper(s) topic should either be:

- Related to your project (can serve as part of your literature review)
- Or some NLP topic your group finds interesting

Paper Presentation: Evaluation

The presentation will be assessed on the student's ability to clearly explain (50%):

- the paper's motivation
- technical approach
- key results

To critically evaluate its assumptions, limitations, and contributions (30%)

To situate the work within the broader LLM and NLP research landscape (20%)

Paper Presentation: Evaluation

Students will also be evaluated on *the quality of the discussion they facilitate*:

- through insightful questions
- engagement with peers
- ability to surface open problems and future research directions

both during their own presentation and when leading discussions for others.

Audience Expectations

- Active participation encouraged (counts toward participation grade)
- Peer review and feedback form for each presentation

Paper Presentation Guidelines

The slides of the presentation need to be shared a day before the presentation to the class.

Please add to the slide deck TA sent the night before, and TA will distribute before the class in the morning.

Paper Review Report Requirement

2-page summary of the paper review

Reviews will be assessed based on answering a small set of questions, to be released at the time of the paper review assignment, clearly and correctly.

In most cases, each question will warrant at minimum a paragraph to answer.

Sign-up & Scheduling

Signup sheet for paper presentation AND final project presentation (released after class)

Provide link(s) to planned paper(s)

Indicate whether topic is project-related

Dates:

- Strict 15 min: 3/25 (Wed), 3/30 (Mon), 4/1 (Wed)
- May extend to ~20 min: 4/13 (Mon), 4/15 (Wed)

Sign-up & Scheduling

Choose a date appropriate for your group size / number of papers

First-come slots (groups may swap dates is mutually agreed)

Paper presentation date + topic due end of week (March 6)

[illegible]

Recap

Issues with Maximization-Based Decoding

- Either greedy or beam search
- Beam search can be more effective with large beam width, but also more expensive
- Another key issue:

Generation can be bland or repetitive
(also called degenerate)

Context:

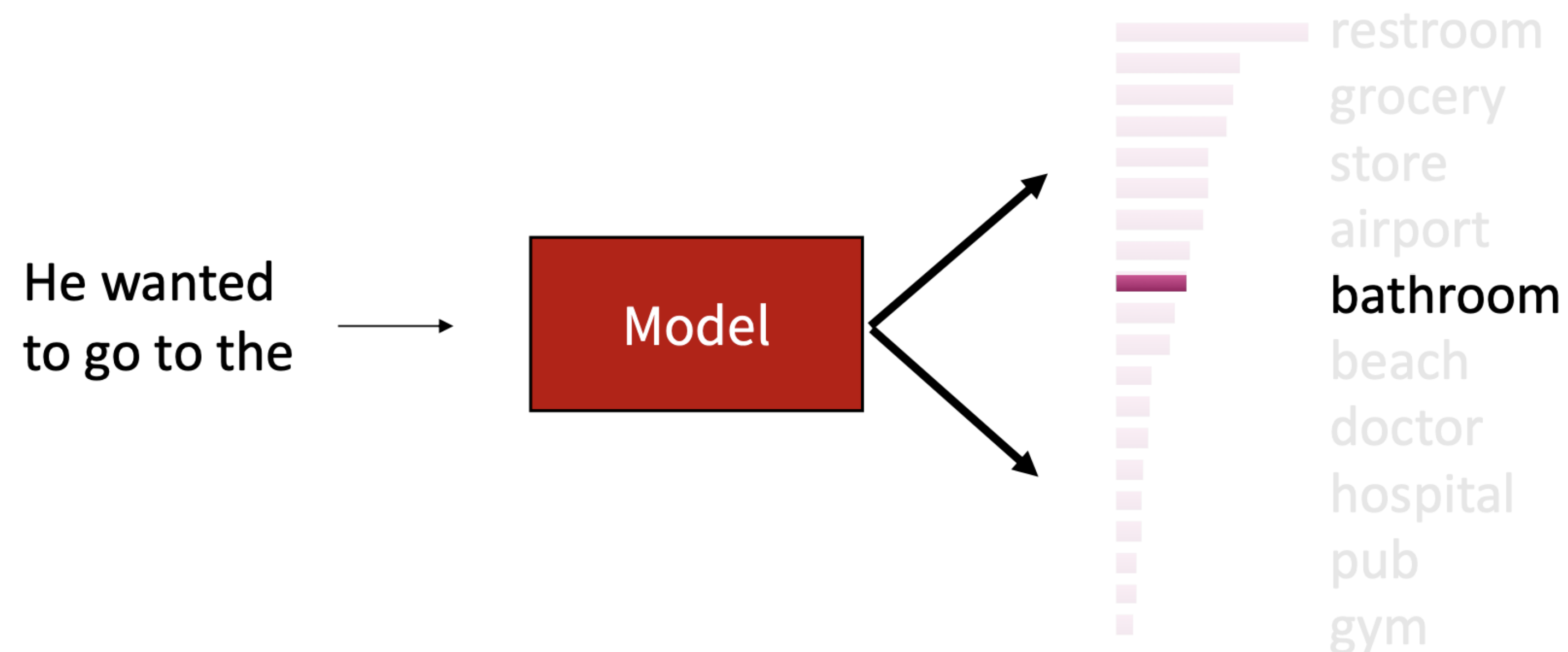
In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.

Continuation:

The study, published in the Proceedings of the National Academy of Sciences of the United States of America (PNAS), was conducted by researchers from the **Universidad Nacional Autónoma de México (UNAM)** and **the Universidad Nacional Autónoma de México (UNAM/Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México/ Universidad Nacional Autónoma de México...**

Solution: Don't Maximize, Pick a Sample

- Sample a token from the distribution of tokens.
- But this is not a random sample, it is a sample for the learned model distribution
 - Many good options which are not the maximum probability!



Pure Sampling

- Sample directly from P_t
- Still has access to the entire vocabulary
- But if the model distributions are of low quality, generations will be of low quality as well
- Often results in ill-formed generations
 - No guarantee of fluency

$$y_t \sim P_t(w) = \frac{\exp(S_w)}{\sum_{v \in V} \exp(S_v)}$$

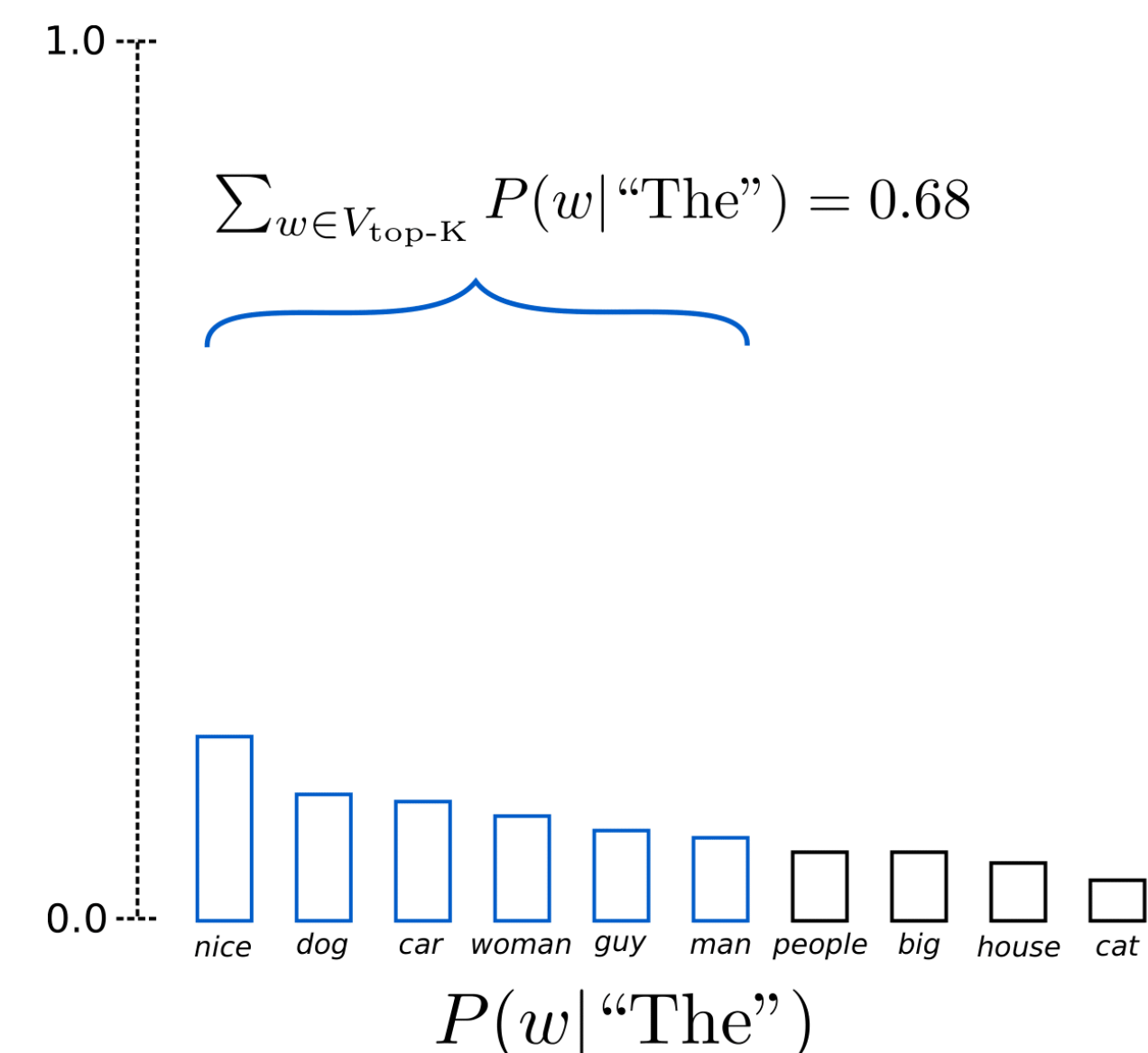
He wanted
to go to the

Model

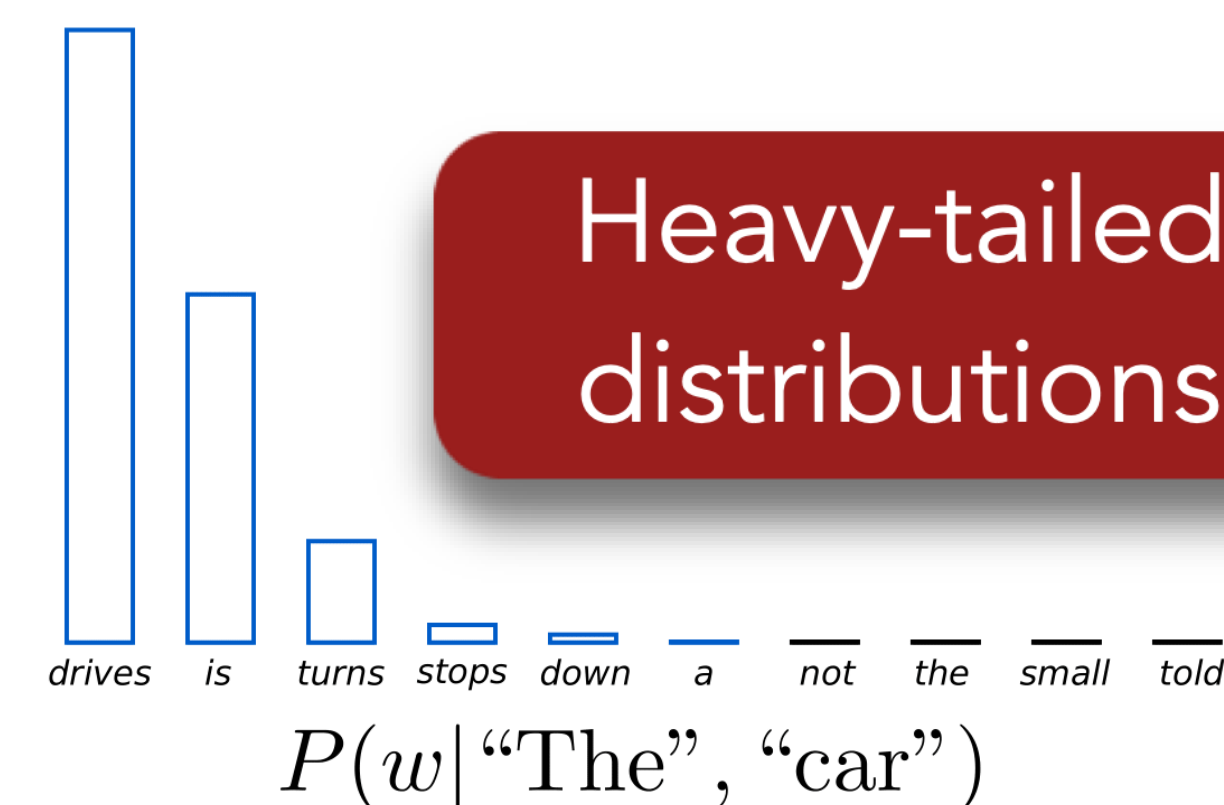
restroom
grocery
store
airport
bathroom
beach
doctor
hospital
pub
gym

Top- K Sampling

- Problem: Pure sampling makes every token in the vocabulary an option
 - Even if most of the probability mass in the distribution is over a limited set of options, the tail of the distribution could be very long and in aggregate have considerable mass
 - Many tokens are probably really wrong in the current context. Yet, we give them individually a tiny chance to be selected.
 - But because there are many of them, we still give them as a group a high chance to be selected.
- Solution: Top- K sampling
 - Only sample from the top K tokens in the probability distribution

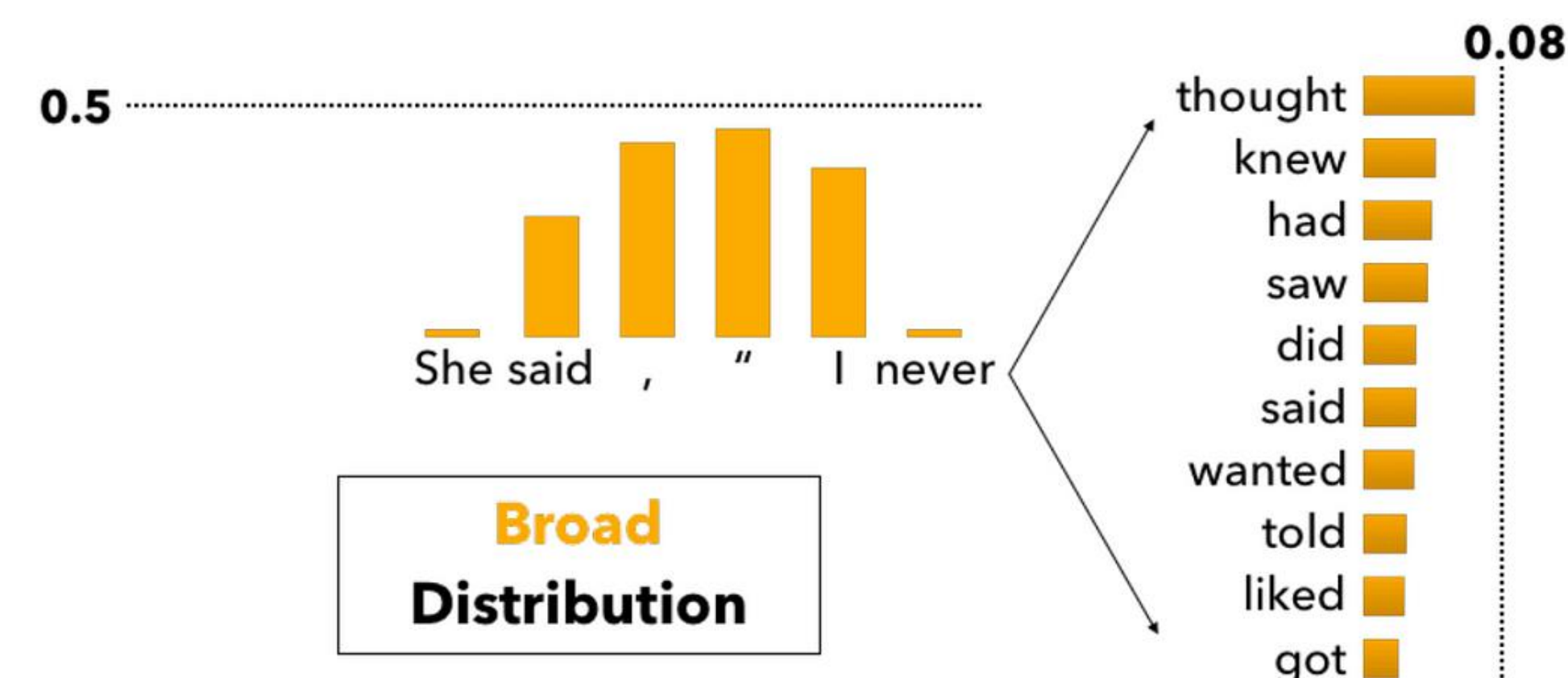


$$\sum_{w \in V_{\text{top-K}}} P(w | \text{"The"}, \text{"car"}) = 0.99$$

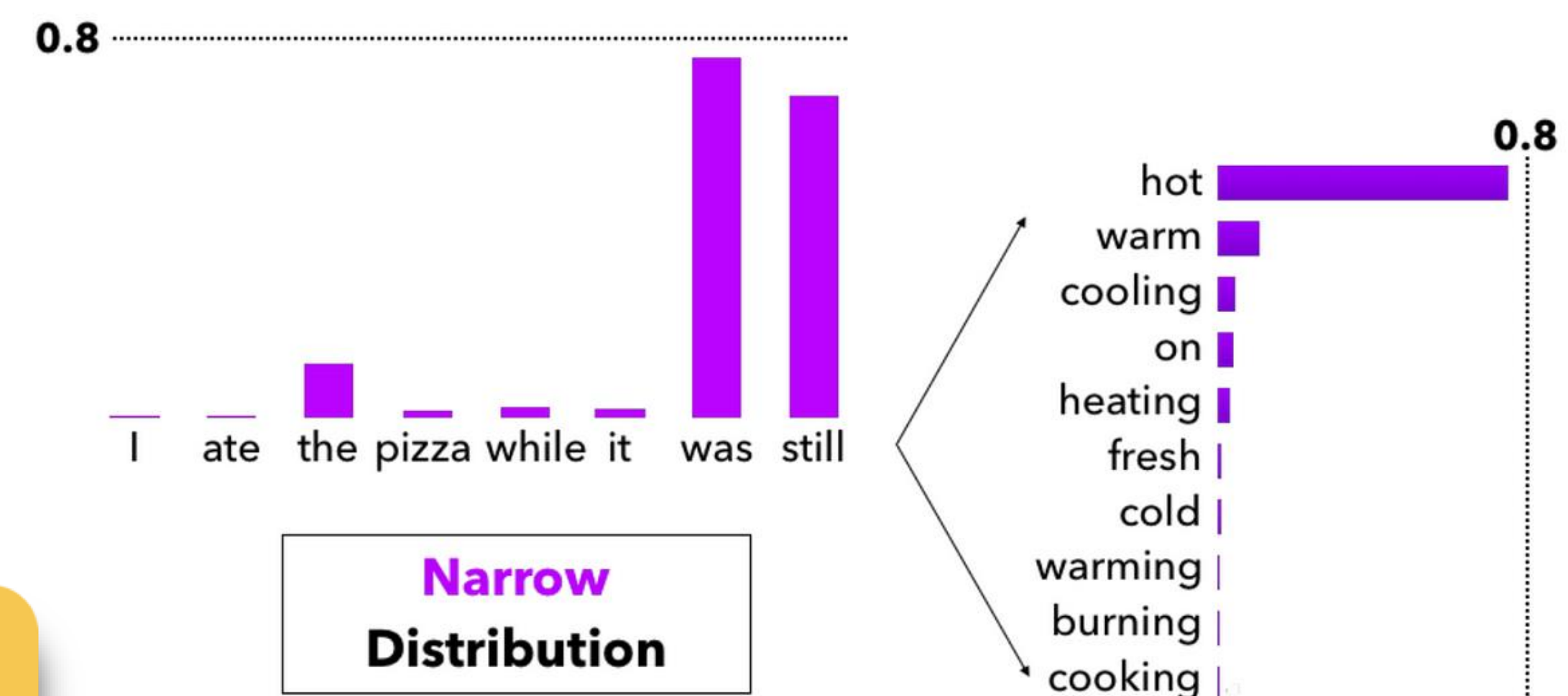


Top- K Sampling: Issues

Top- K sampling can cut off too quickly



Top- K sampling can also cut off too slowly!

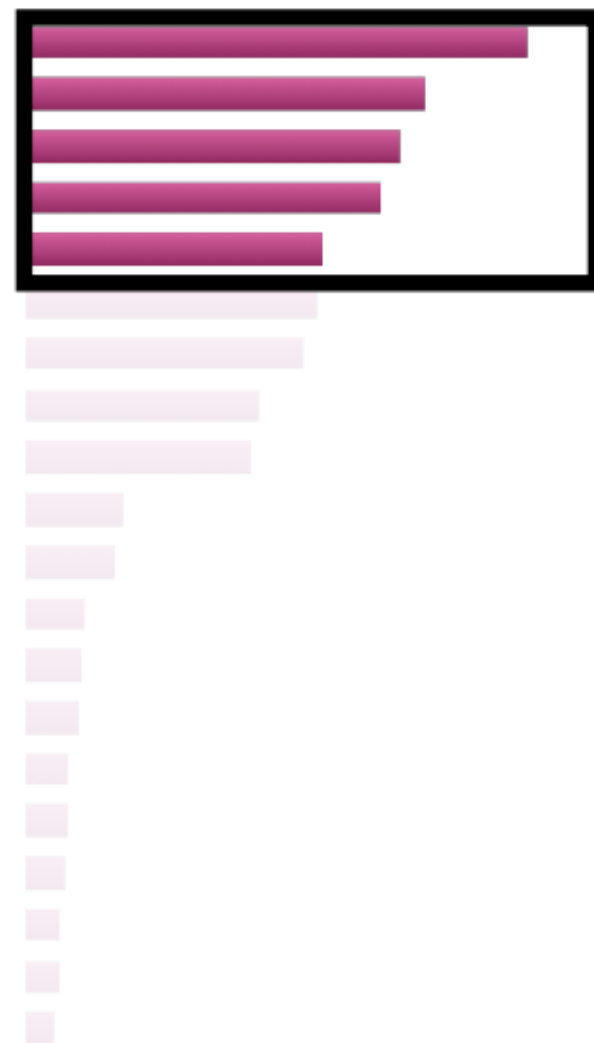


We can do better than having one-size-fits-all: a fixed K for all contexts

Nucleus (Top- P) Sampling

- Solution: Top- P sampling
 - Sample from all tokens in the top P cumulative probability mass (i.e., where mass is concentrated)
 - Varies K depending on the uniformity of P_t

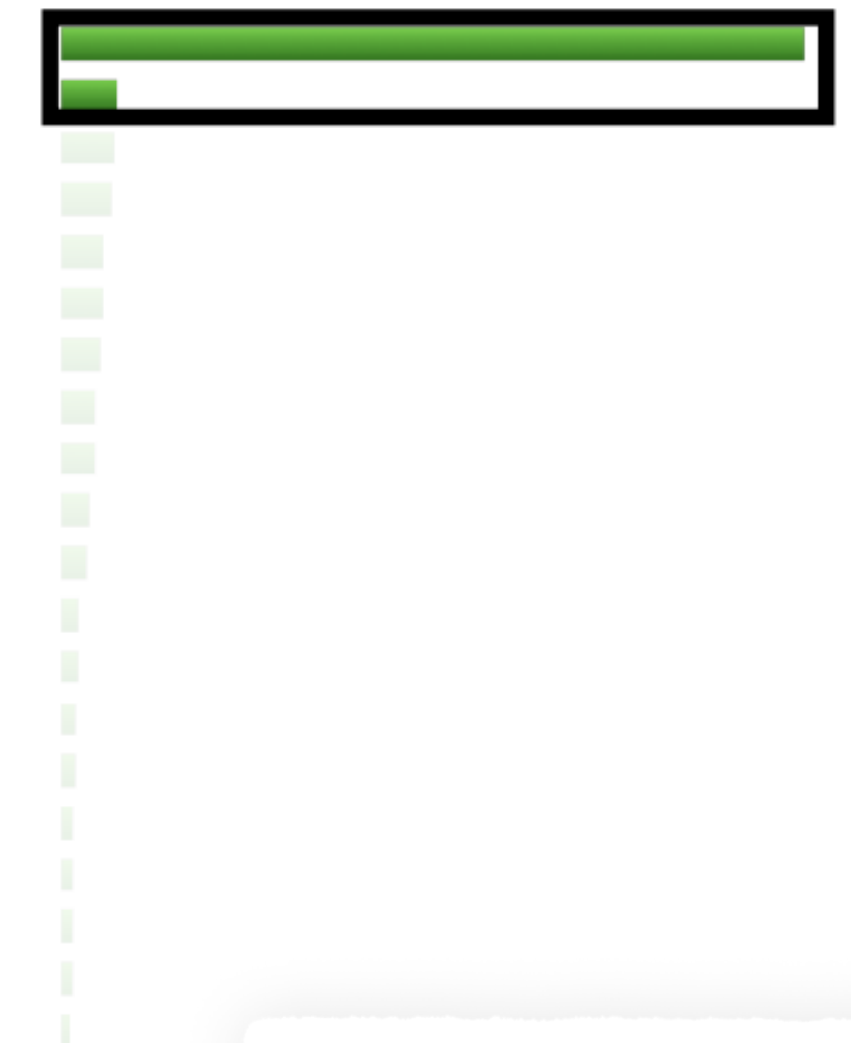
$$P_t^1(y_t = w | \{y\}_{<t})$$



$$P_t^2(y_t = w | \{y\}_{<t})$$



$$P_t^3(y_t = w | \{y\}_{<t})$$



Temperature Scaling

$$P(y_t = w) = \frac{\exp(S_w)}{\sum_{v \in V} \exp(S_v)}$$

- Recall: On timestep t , the model computes a prob distribution P_t by applying the softmax function to a vector of scores $S \in \mathbb{R}^{|V|}$
- We can apply a temperature hyperparameter τ to the softmax to rebalance P_t

$$P(y_t = w) = \frac{\exp(S_w/\tau)}{\sum_{v \in V} \exp(S_v/\tau)}$$

- Raise the temperature $\tau > 1$: P_t becomes more uniform
 - More diverse output (probability is spread around vocab)
- Lower the temperature $\tau < 1$: P_t becomes more spiky
 - Less diverse output (probability is concentrated on top words)

Temperature is a hyperparameter for decoding: It can be tuned for both beam search and sampling.

Three Stages of Training LLMs

Stage 1

- Pre-training on large corpus of text
- Produces a Base Language Model
- Continued Pre-training for domain adaptation (optional; sometimes called mid-training, stage 1.5)

Stage 2

- Post-training for Task Adaptation
- Seq2Seq Instruction Tuning (Supervised Finetuning, different meaning than BERT-style classification tasks)

Stage 3

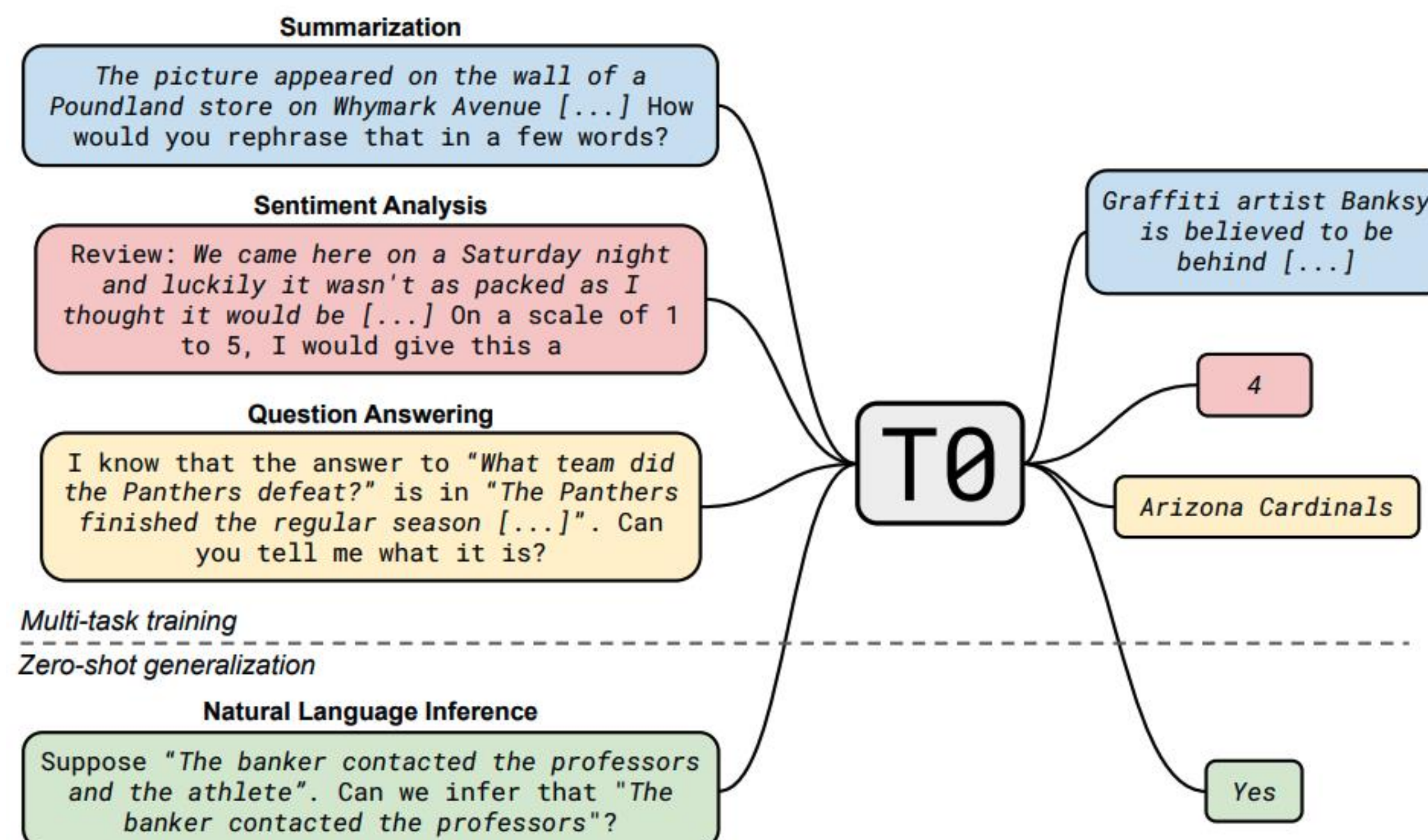
- Post-training for Preference Alignment
 - Either Reinforcement Learning with Human Feedback : Train a supervised classifier (reward model) on human demonstrations to provide feedback to LM. Reinforcement learning to maximize rewards given by reward model
 - Or, train with a preferred and a dispreferred generation (more popular now)

Inference

Prompting with Instructions and Demonstrations (also called examples, shots)

Instruction Tuning

- Pretraining:
 - Train a model to continue a given context
- Instruction Tuning:
 - Train a model to follow varied instructions
 - Needed because the vast majority of pretraining is done on data which are not in the form of instructions
 - Fine-tuned (using the next-token-prediction objective) on a dataset of instructions together with correct responses



"Multitask Prompted Training Enables Zero-Shot Task Generalization" (Sahn et al., 2022)

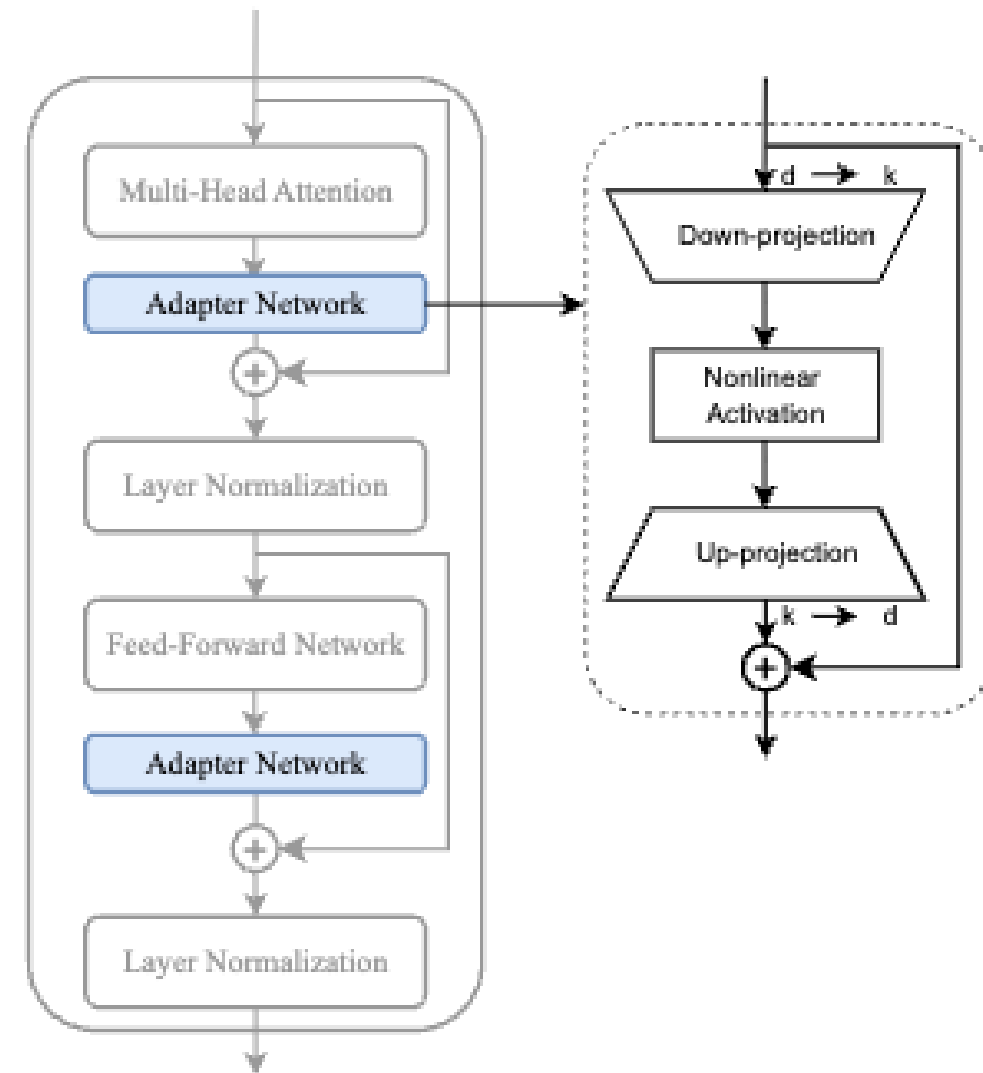
Instruction Tuning Data

- Instruction tuning datasets are often created by repurposing standard NLP datasets for tasks like question answering or machine translation
- Often synthesized!
- Prompting existing LLMs
- More variety in the instruction templates → better models!

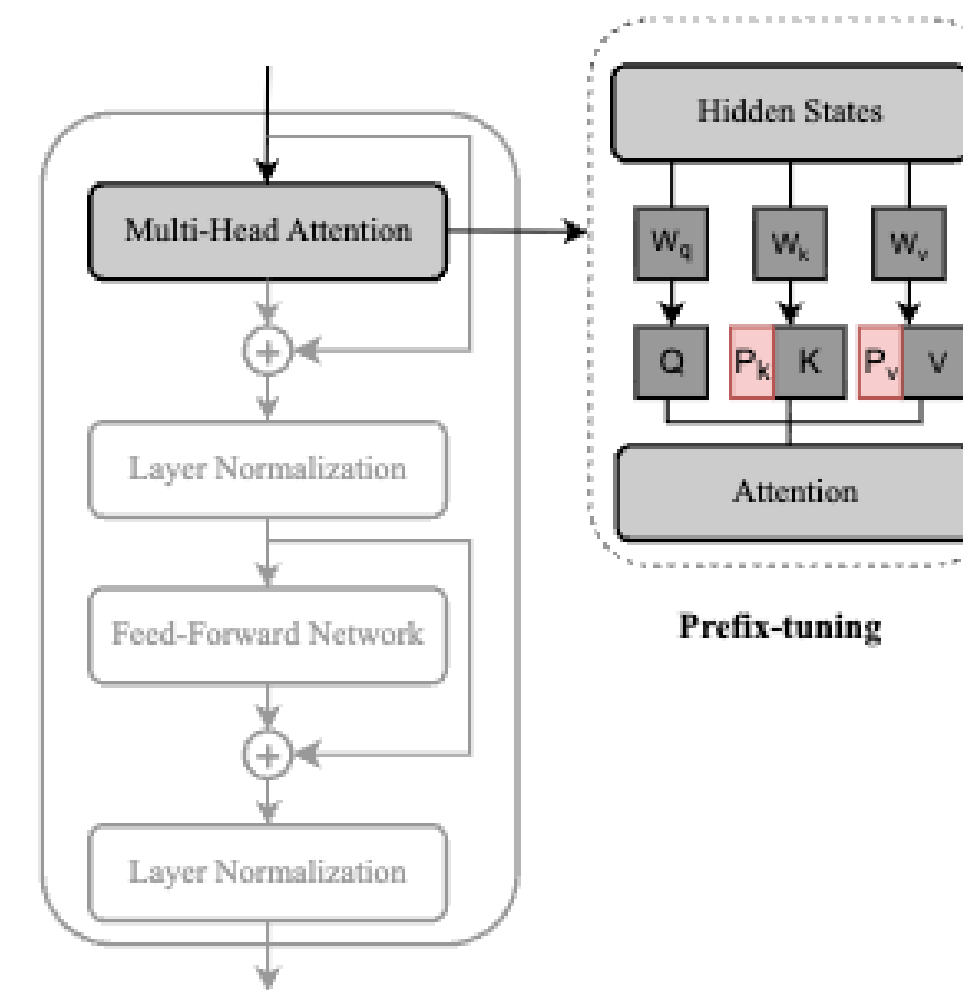
Release	Collection	Model	Model Details			Data Collection & Training Details			
			Base	Size	Public?	Prompt Types	Tasks in Flan	# Exs	Methods
2020 05	UnifiedQA	UnifiedQA	RoBerta	110-340M	P	ZS	46 / 46	750k	
2021 04	CrossFit	BART-CrossFit	BART	140M	NP	FS	115 / 159	71M	
2021 04	Natural Inst v1.0	Gen. BART	BART	140M	NP	ZS / FS	61 / 61	620k	+ Detailed k-shot Prompts
2021 09	Flan 2021	Flan-LaMDA	LaMDA	137B	NP	ZS / FS	62 / 62	4.4M	+ Template Variety
2021 10	P3	T0, T0+, T0++	T5-LM	3-11B	P	ZS	62 / 62	12M	+ Template Variety + Input Inversion
2021 10	MetalCL	MetalCL	GPT-2	770M	P	FS	100 / 142	3.5M	+ Input Inversion + Noisy Channel Opt
2021 11	ExMix	ExT5	T5	220M-11B	NP	ZS	72 / 107	500k	+ With Pretraining
2022 04	Super-Natural Inst.	Tk-Instruct	T5-LM, mT5	11-13B	P	ZS / FS	1556 / 1613	5M	+ Detailed k-shot Prompts + Multilingual
2022 10	GLM	GLM-130B	GLM	130B	P	FS	65 / 77	12M	+ With Pretraining + Bilingual (en, zh-cn)
2022 11	xP3	BLOOMz, mT0	BLOOM, mT5	13-176B	P	ZS	53 / 71	81M	+ Massively Multilingual
2022 12	Unnatural Inst. [†]	T5-LM-Unnat. Inst.	T5-LM	11B	NP	ZS	~20 / 117	64k	+ Synthetic Data
2022 12	Self-Instruct [†]	GPT-3 Self Inst.	GPT-3	175B	NP	ZS	Unknown	82k	+ Synthetic Data + Knowledge Distillation
2022 12	OPT-IML Bench [†]	OPT-IML	OPT	30-175B	P	ZS + FS CoT	~2067 / 2207	18M	+ Template Variety + Input Inversion + Multilingual
2022 10	Flan 2022 (ours)	Flan-T5, Flan-PaLM	T5-LM, PaLM	10M-540B	P NP	ZS + FS CoT	1836	15M	+ Template Variety + Input Inversion + Multilingual

"The Flan Collection: Designing Data and Methods for Effective Instruction Tuning" (Longpre et al., 2023)

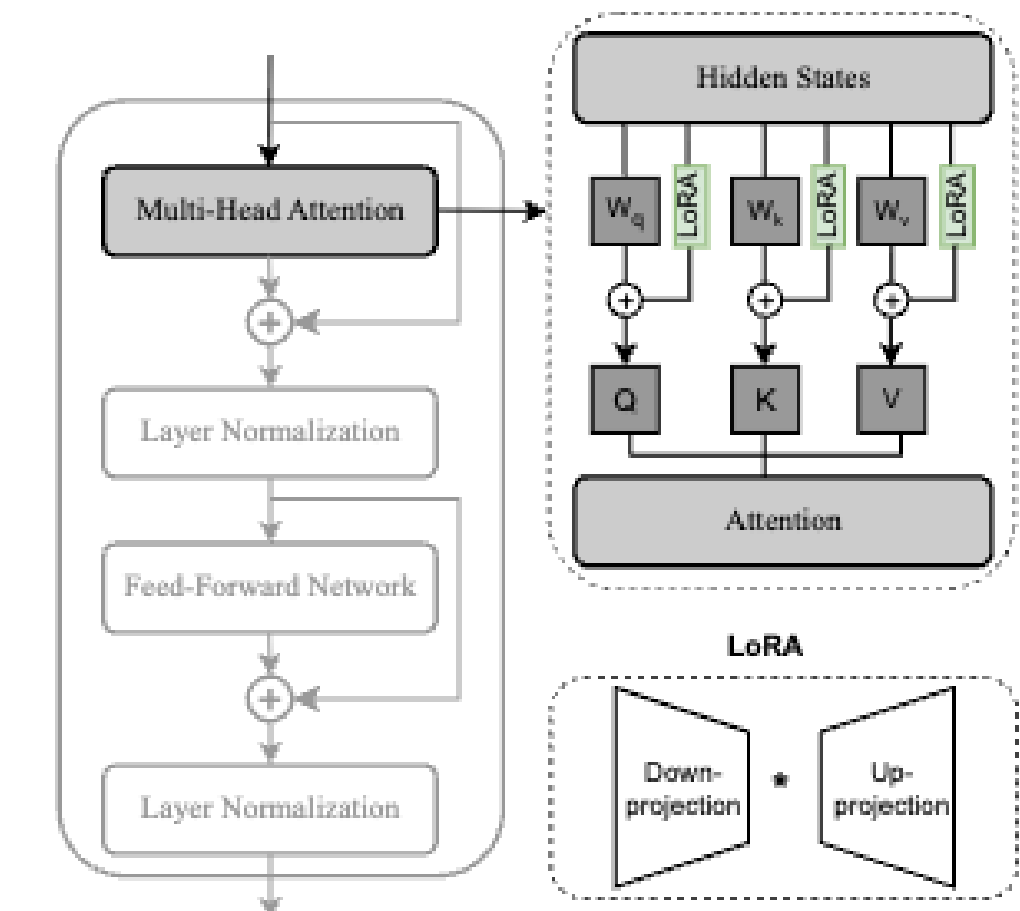
Parameter-Efficient Fine-tuning



(a) Sequential Adapter



(b) Prefix-tuning



(c) LoRA

- Fine-tuning can be very difficult and expensive with LLMs
 - enormous numbers of parameters to train; think 70B parameters, and their gradients!!
 - each pass of batch gradient descent has to backpropagate through many many huge layers.
- Alternative: allow a model to be finetuned without changing all the parameters.
 - parameter-efficient fine tuning or PEFT,
 - efficiently select a subset of parameters to update when finetuning and keep the rest frozen
- Examples: Adapters, Prefix-Tuning, LoRA or Low Rank Adaptation

LoRA: Low-Rank Adaptation

- Instead of updating weight matrices in attention layers during finetuning, Low-Rank Adaptation eases this by updating a low-rank approximation of the matrix : matrices which are far smaller
- LoRA freezes the pre-trained model weights and injects trainable rank decomposition matrices into each layer of the Transformer architecture
- Gradient updates are reparametrized as $\mathbf{W}_0 + \Delta\mathbf{W} = \mathbf{W}_0 + \mathbf{B} \cdot \mathbf{A}$
 - Where $\mathbf{B}$ and $\mathbf{A}$ are low-rank matrices, the only matrices to be updated
- LoRA can reduce the number of trainable parameters by 10,000 times and the GPU memory requirement by 3 times for GPT-3 175B
- Usually comes at a (small) cost to performance

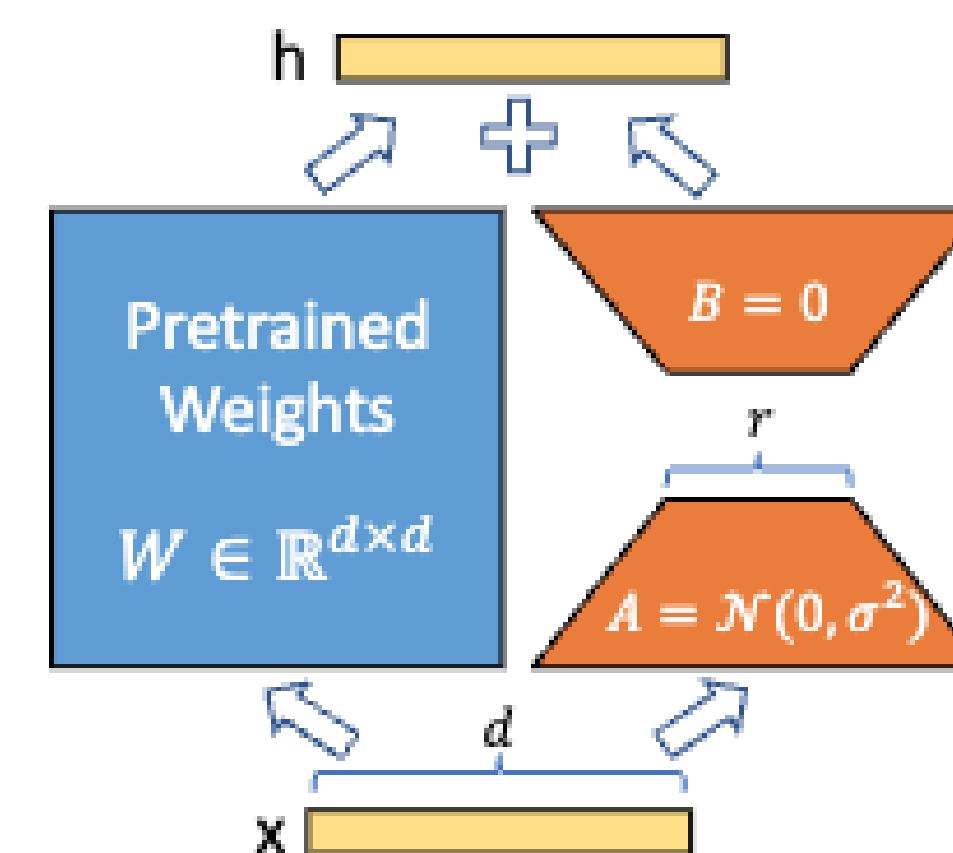
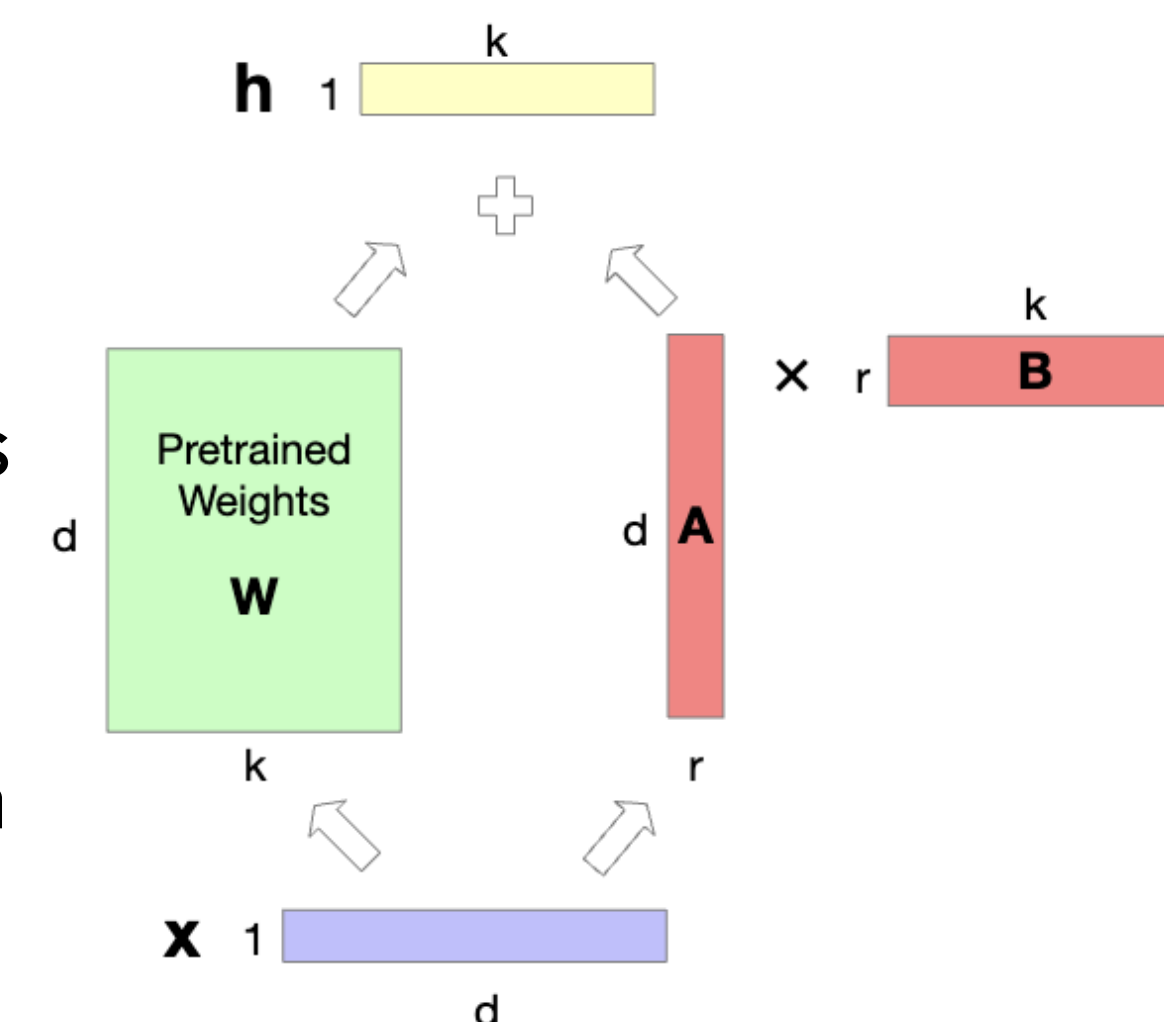


Figure 1: Our reparametrization. We only train A and B .

LoRA. [Hu et al., 2021](#)

Interacting with LLMs: Prompting

Interfacing with Large Language Models

- Once trained, language models can be very powerful
- Prompting (or In-Context / Few-Shot Learning): the ability to do many tasks with no gradient updates and no / a few examples, by simply:
 - You can get interesting zero-shot behavior if you're creative enough with how you specify your task!

Basic Prompt Templates

Summarization	<code>{input} ; tldr;</code>
Translation	<code>{input} ; translate to French:</code>
Sentiment	<code>{input}; Overall, it was</code>
Fine-Grained-Sentiment	<code>{input}; What aspects were important in this review?</code>

Prompting

- Interface to a language model: prompts in natural language
- Very large language models seem to perform some kind of “learning” without gradient updates!!! “Learn” simply from examples you provide within their contexts
 - Sometimes called in-context learning
 - Misnomer: no learning (parameter update) actually happens during prompting
- Can be zero shot (without examples) or few-shot (with a few examples)
 - Typically < 10
 - With powerful LLMs, 0-shot approaches are all you need!

```
1  Translate English to French:  ← task description
2  cheese => .....           ← prompt
```

Zero-Shot

```
1  Translate English to French:  ← task description
2  sea otter => loutre de mer     ← examples
3  peppermint => menthe poivrée  ←
4  plush girafe => girafe peluche ←
5  cheese => .....             ← prompt
```

Few-Shot

“Language Models are Few-Shot Learners” (Brown et al., 2020)

Prompting: Success

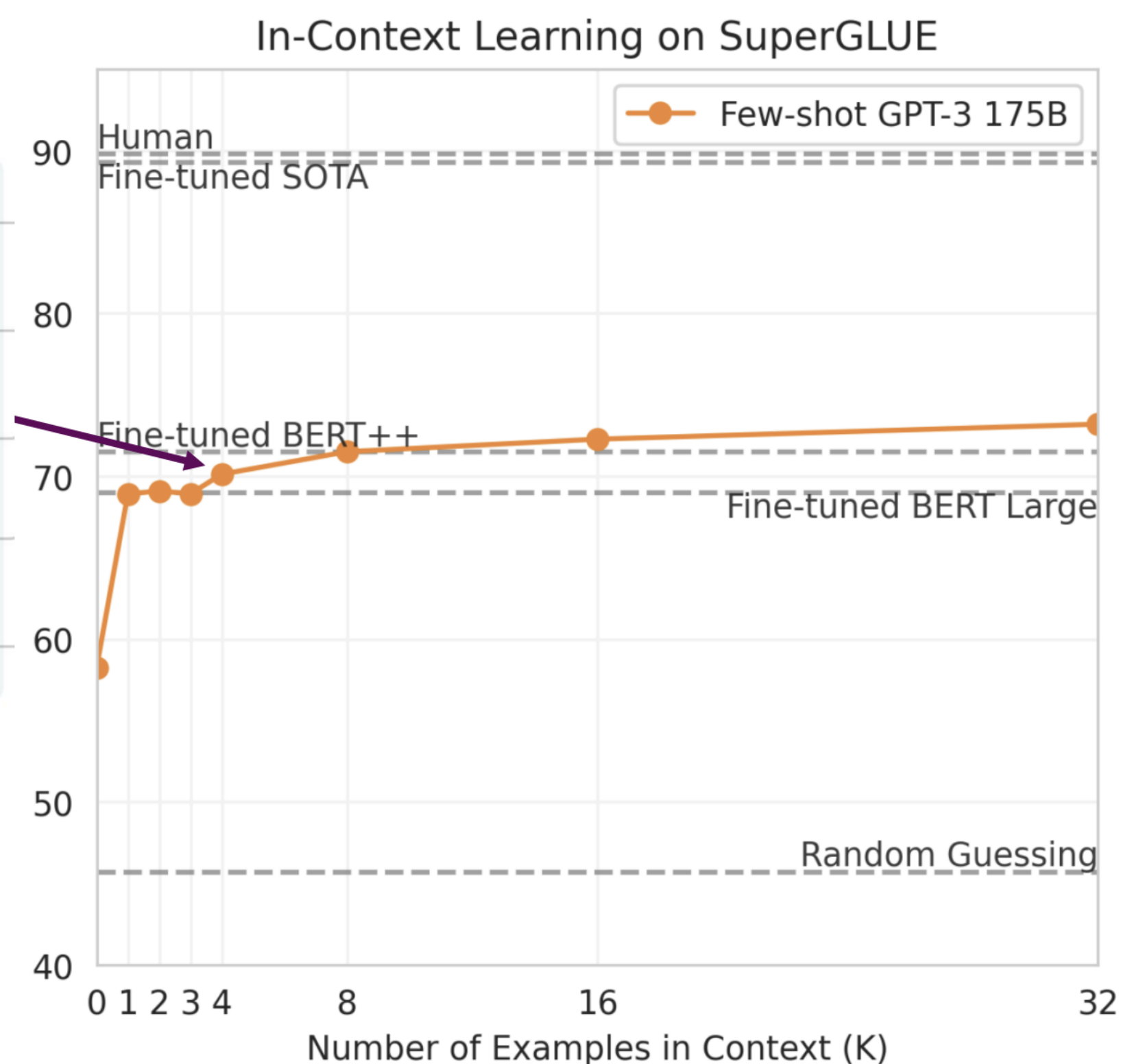
- Much more flexible than older formulation of pre-training encoder-only models and fine-tuning to specific classification tasks (the BERT paradigm)
- Now, pre-train and instruction tune one large model and prompt it to do a variety of tasks!

Emergent few-shot learning

Few-shot

```

1 Translate English to French:
2 sea otter => loutre de mer
3 peppermint => menthe poivrée
4 plush girafe => girafe peluche
5 cheese => .....
  
```



[Brown et al., 2020]

Why does prompting work so well?

- Induction heads
- Discovered by looking at mini language models with only 1-2 attention heads
- If the model sees the pattern $AB \dots A$ in an input sequence, it predicts that B will follow, instantiating the pattern completion rule $AB \dots A \rightarrow B$
- Perhaps a generalized fuzzy version of this pattern completion rule, implementing a rule like $A^*B^* \dots A \rightarrow B^*$, where $A^* \approx A$ and $B^* \approx B$ (by $\approx$ we mean some form of semantic similarity), might be responsible for in-context learning

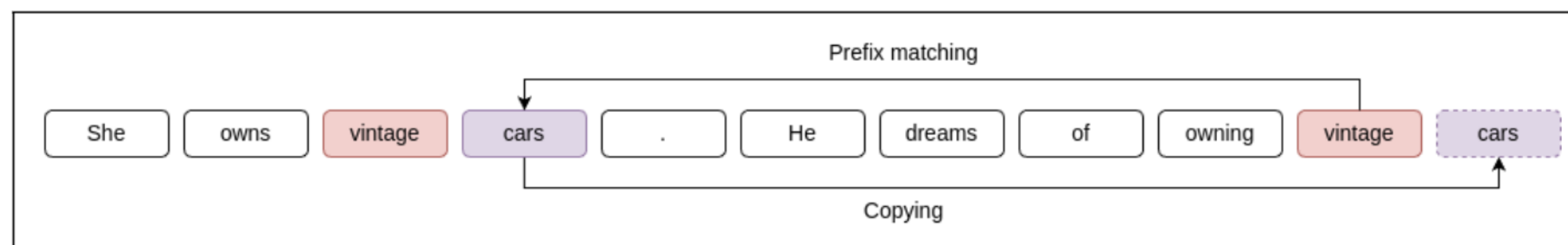
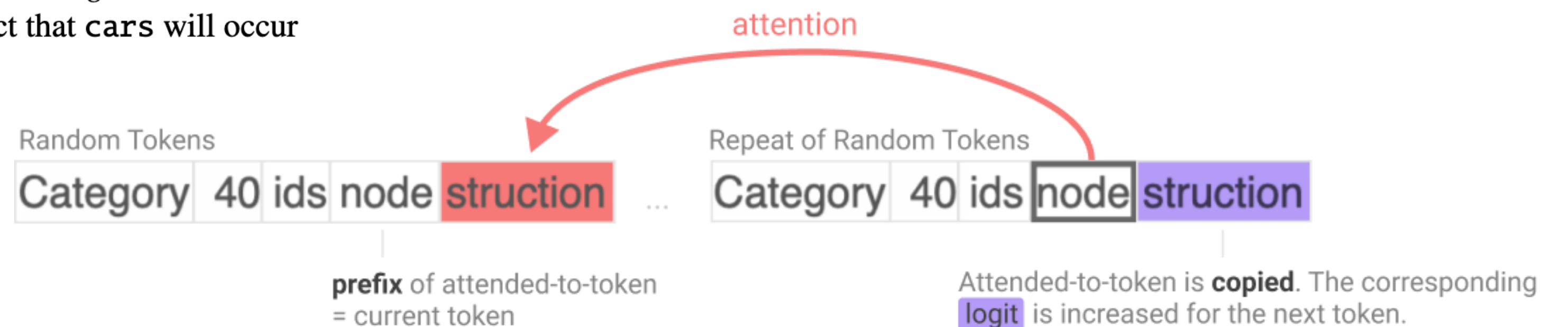


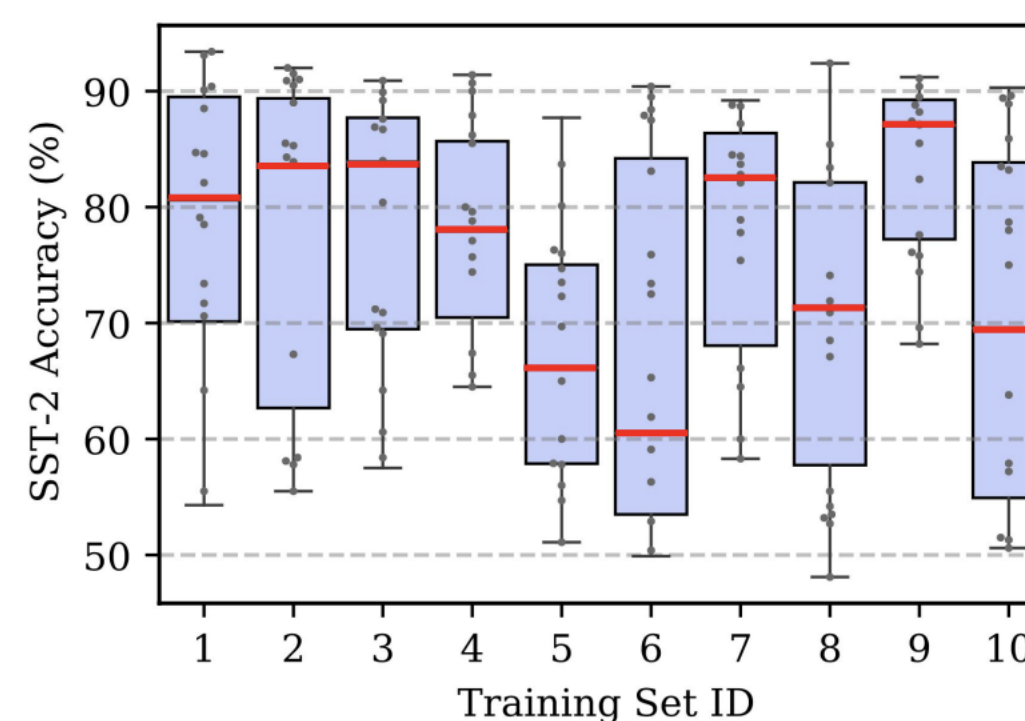
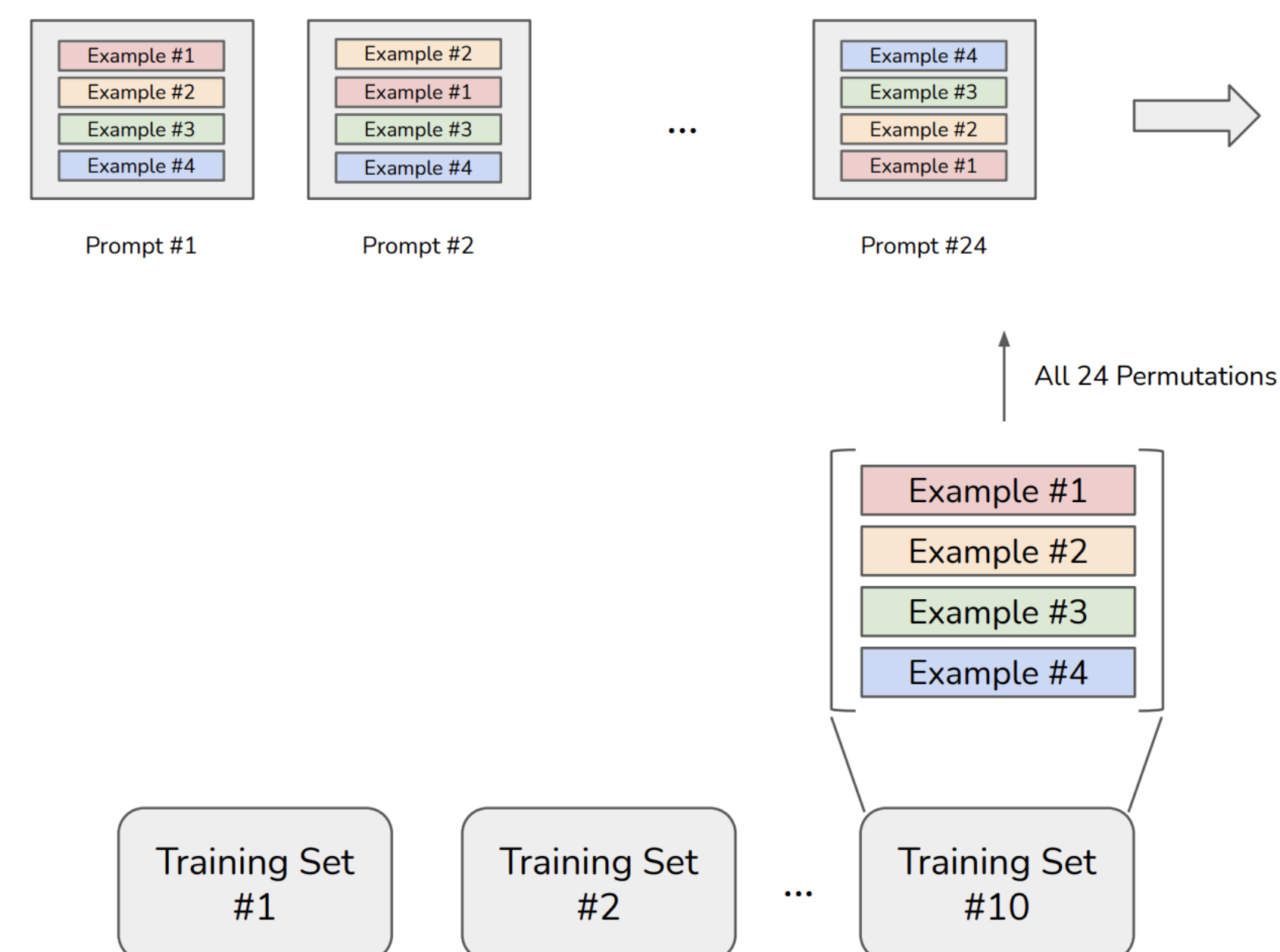
Figure 12.3 An induction head looking at **vintage** uses the *prefix matching* mechanism to find a prior instance of **vintage**, and the *copying* mechanism to predict that **cars** will occur again. Figure from [Crosbie and Shutova \(2022\)](#).



"In-context Learning and Induction Heads" (Olsson et al., 2022)

Prompting Limitations: Prompt Design

- Task performance is sensitive to prompt design
 - Formatting
 - Ordering of demonstrations
 - Wording of the prompt



“Calibrate Before Use:
Improving Few-Shot
Performance of Language
Models” (Zhao et al., 2021)

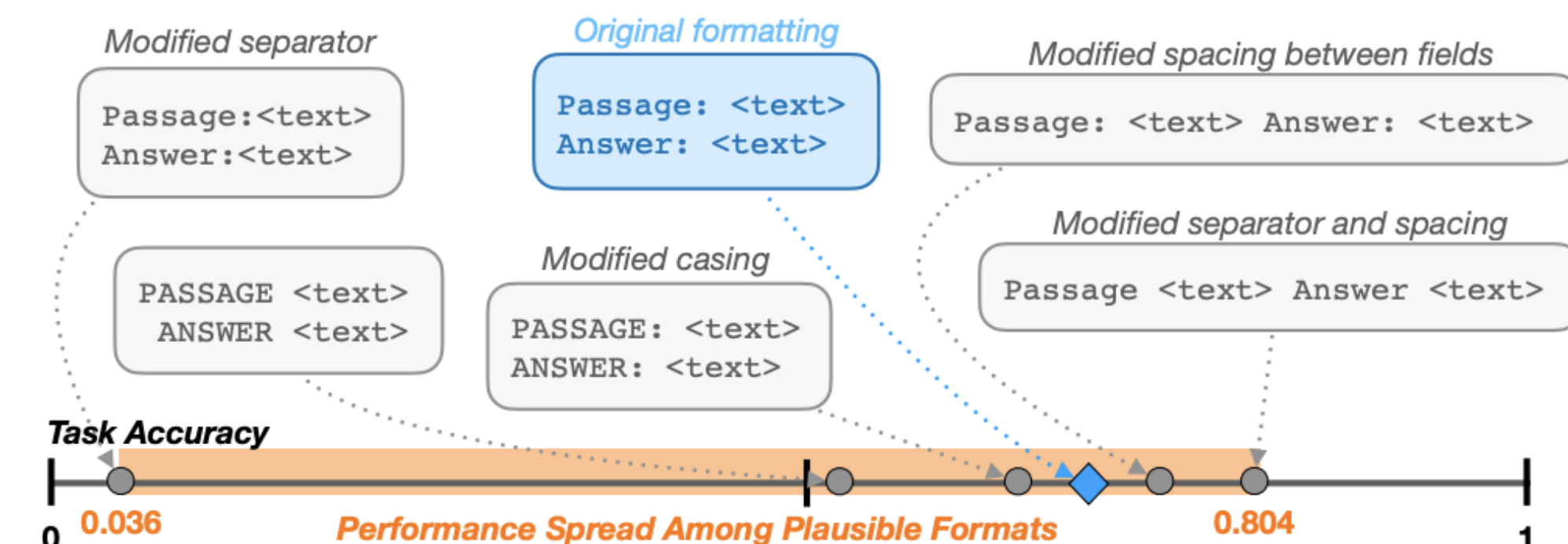


Figure 1: Slight modifications in prompt format templating may lead to significantly different model performance for a given task. Each <text> represents a different variable-length placeholder to be replaced with actual data samples. Example shown corresponds to 1-shot LLaMA-2-7B performances for task280 from SuperNaturalInstructions (Wang et al., 2022). This StereoSet-inspired task (Nadeem et al., 2021) requires the model to, given a short passage, classify it into one of four types of stereotype or anti-stereotype (gender, profession, race, and religion).

Sclar et al., ICLR 2024

Chain-of-Thought Prompting

- Since the model is trained on lots and lots of language data, perhaps relying on its capabilities to generate language can make it more accurate!

Standard Prompting

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The answer is 27. ❌

Zero-Shot Chain-of-Thought Prompting

- The model may not even need examples of reasoning, it may be able to “reason” on its own if provided the right trigger context

Model Input

Q: Roger has 5 tennis balls. He buys 2 more cans of tennis balls. Each can has 3 tennis balls. How many tennis balls does he have now?

A: Roger started with 5 balls. 2 cans of 3 tennis balls each is 6 tennis balls. $5 + 6 = 11$. The answer is 11.

Q: The cafeteria had 23 apples. If they used 20 to make lunch and bought 6 more, how many apples do they have?

Model Output

A: The cafeteria had 23 apples originally. They used 20 to make lunch. So they had $23 - 20 = 3$. They bought 6 more apples, so they have $3 + 6 = 9$. The answer is 9. ✓

Q: A juggler can juggle 16 balls. Half of the balls are golf balls, and half of the golf balls are blue. How many blue golf balls are there?

A: **Let's think step by step.** There are 16 balls in total. Half of the balls are golf balls. That means there are 8 golf balls. Half of the golf balls are blue. That means there are 4 blue golf balls. ✓

Chain-of-Thoughts Performance

	MultiArith	GSM8K
Zero-Shot	17.7	10.4
Few-Shot (2 samples)	33.7	15.6
Few-Shot (8 samples)	33.8	15.6
Zero-Shot-CoT	Greatly outperforms zero-shot → 78.7	40.7
Few-Shot-CoT (2 samples)	84.8	41.3
Few-Shot-CoT (4 samples : First) (*1)	89.2	-
Few-Shot-CoT (4 samples : Second) (*1)	90.5	-
Few-Shot-CoT (8 samples)	Manual CoT → 93.0	48.7
	still better	

Kojima et al., 2022

Zero-shot CoT Trigger Prompt	Accuracy
Let's work this out in a step by step way to be sure we have the right answer.	82.0
Let's think step by step. (*1)	78.7
First, (*2)	77.3
Let's think about this logically.	74.5
Let's solve this problem by splitting it into steps. (*3)	72.2
Let's be realistic and think step by step.	70.8
Let's think like a detective step by step.	70.3
Let's think	57.5
Before we dive into the answer,	55.7
The answer is after the proof.	45.7
(Zero-shot)	17.7

There seems to be some wiggle room in the exact prompt to be used for achieving the best performance!

Preference Tuning: Aligning Models with Human Preferences

The need for preference alignment

A Pre-trained GPT-3

Prompt: Explain the moon landing to a six year old in a few sentences.

Output: Explain the theory of gravity to a 6 year old.

Prompt: Translate to French: The small dog

Output: The small dog crossed the road.

Ouyang et al., 2022; J&M Chap 12

- Make LLMs more helpful
 - Supervised Finetuning: Instruction Tuning
 - Prompting
- Make LLMs less harmful
 - Avoid unsafe responses which may cause harm (e.g. privacy)
- Model alignment with human preferences aims to achieve both!
 - Algorithms may involve reinforcement learning (such as PPO, GRPO) or not (DPO)

Preference Alignment

- Let's say we were training a language model on some task (e.g. summarization).
- For an instruction x and a LM sample y , imagine we had a way to obtain a human reward of that summary: $R(x, y) \in \mathbb{R}$, higher is better.

SAN FRANCISCO,
California (CNN) --
A magnitude 4.2
earthquake shook the
San Francisco
...
overturn unstable
objects.

x

An earthquake hit
San Francisco.
There was minor
property damage,
but no injuries.

$$y_1$$

$$R(x, y_1) = 8.0$$

The Bay Area has
good weather but is
prone to
earthquakes and
wildfires.

$$y_2$$

$$R(x, y_2) = 1.2$$

- Maximize the expected reward of samples from our LM: $\mathbb{E}_{y \sim p_\theta(y|x)} [RM_\phi(x, \hat{y})]$

Step 1

Collect demonstration data and train a supervised policy.

A prompt is sampled from our prompt dataset.

↻
Explain reinforcement learning to a 6 year old.

A labeler demonstrates the desired output behavior.

👤
We give treats and punishments to teach...

This data is used to fine-tune GPT-3.5 with supervised learning.

SFT
🧠
📄📄📄

Instruction Tuning!

Step 2

Collect comparison data and train a reward model.

A prompt and several model outputs are sampled.

↻
Explain reinforcement learning to a 6 year old.

A B
In reinforcement learning, the agent is... Explain rewards...
C D
In machine learning... We give treats and punishments to teach...

A labeler ranks the outputs from best to worst.

👤
D > C > A > B

This data is used to train our reward model.

RM
🧠
D > C > A > B

Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

A new prompt is sampled from the dataset.

🦎
Write a story about otters.

The PPO model is initialized from the supervised policy.

PPO
🧠

The policy generates an output.

Once upon a time...

The reward model calculates a reward for the output.

RM
🧠

The reward is used to update the policy using PPO.

r_k

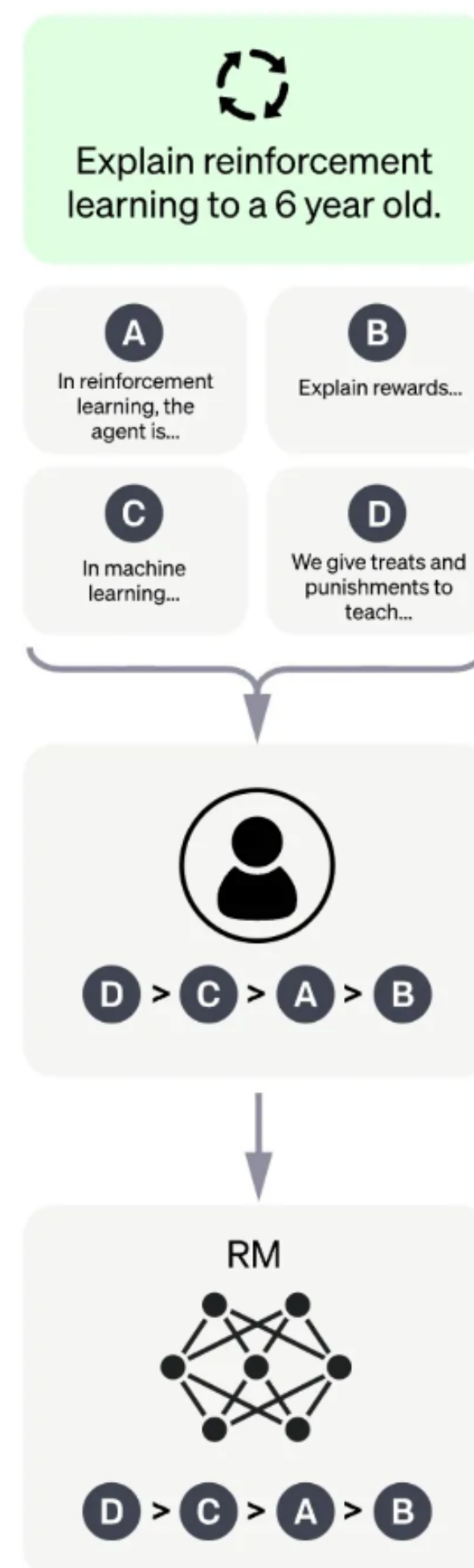
Step 2

Collect comparison data and train a reward model.

A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

This data is used to train our reward model.



Preference Data

- Getting on-the-fly annotations with a human-in-the-loop is expensive!
 - Instead of directly asking humans for preferences, model their preferences as a separate (classification / regression) problem!
- Human judgments are noisy and miscalibrated!
 - Instead of asking for direct ratings, ask for pairwise comparisons, which can be more reliable
 - Nowadays, mostly LLM judges
- Train a reward model, $RM_{\phi}(x, y)$ to predict human reward from an annotated dataset
 - Pairwise preferences converted into scores

Reward Modeling

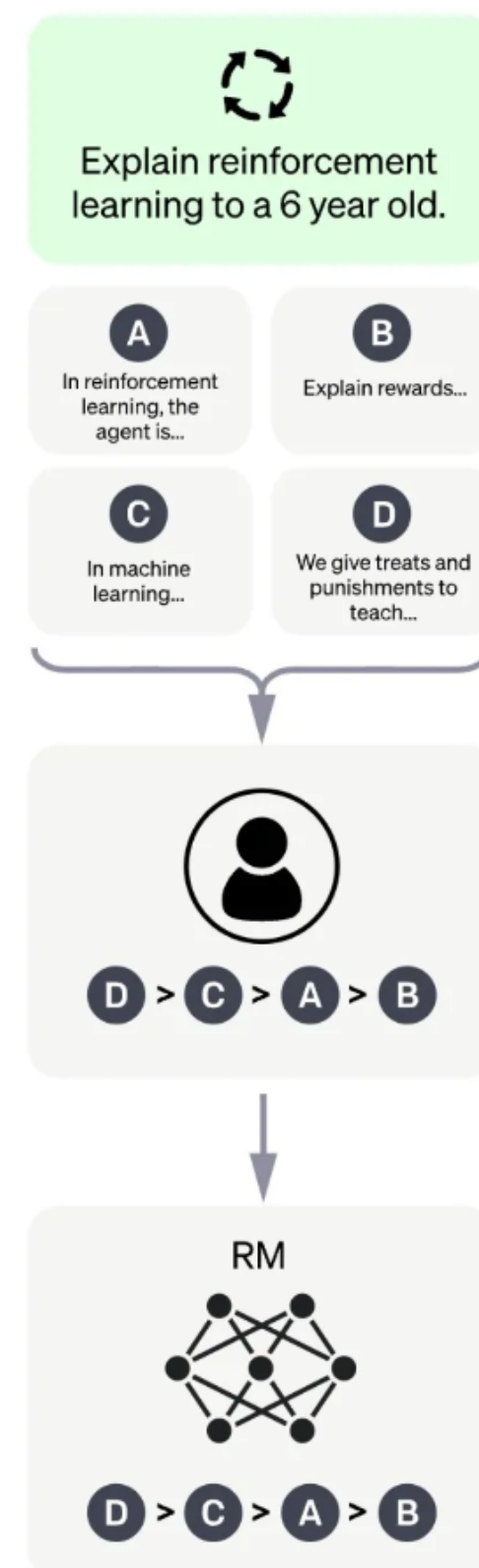
Step 2

Collect comparison data and train a reward model.

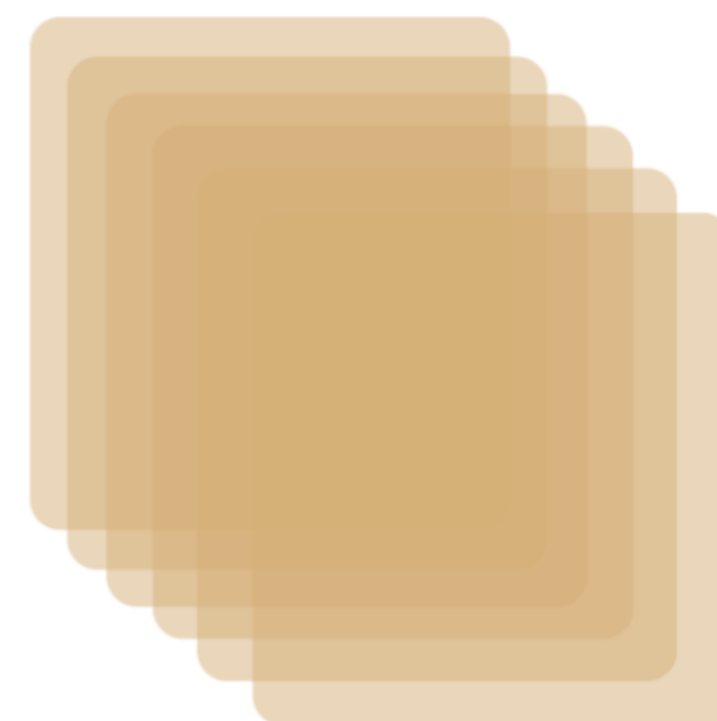
A prompt and several model outputs are sampled.

A labeler ranks the outputs from best to worst.

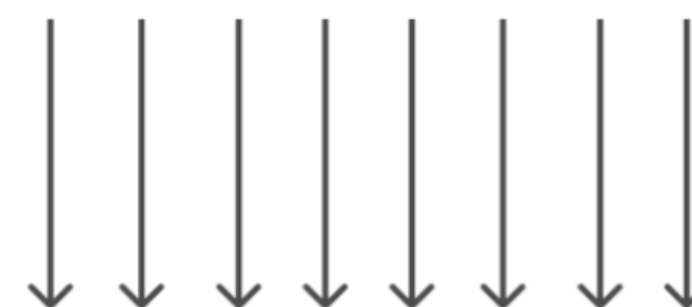
This data is used to train our reward model.



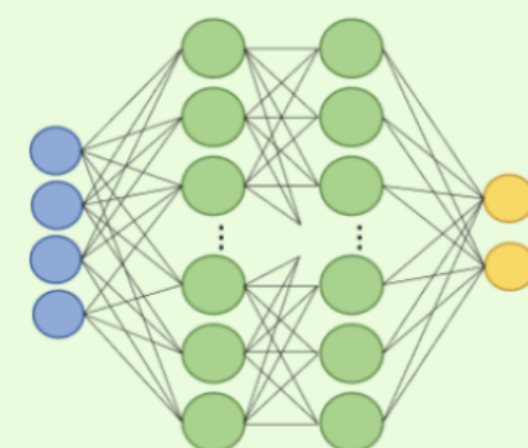
Prompts Dataset



Sample many prompts

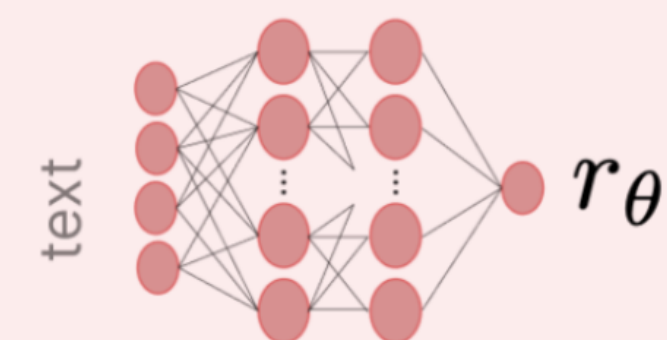


Initial Language Model



Train on {sample, reward} pairs

Reward (Preference) Model



Outputs are ranked (relative, ELO, etc.)



Lorem ipsum dolor
sit amet, consectetur
adipiscing elit. Aenean
Donec quam felis
vulputate eget, arcu
Nam quam nunc
eros faucibus tincidunt
luctus pulvinar, hend

Generated text



Reinforcement Learning with Human Feedback

- Ingredients

- An instruction-tuned LM $p^{SFT}(\hat{y}|x)$
- A reward model $RM_{\phi}(x, y)$

- Step 3 involves:

- Copy the model to $p_{\theta}^{RL}(\hat{y}|x)$
- Optimize: $\mathbb{E}_{\hat{y} \sim p_{\theta}^{RL}(\hat{y}|x)} [RM_{\phi}(x, y)]$

- But, we still want a good instruction-tuned model, not just a reward maximizer

- Hence, we add a penalty for drifting too far from the initialization: $\mathbb{E}_{\hat{y} \sim p_{\theta}^{RL}(\hat{y}|x)} [RM_{\phi}(x, y) -$

$$\beta \log \frac{p_{\theta}^{RL}(\hat{y}|x)}{p^{SFT}(\hat{y}|x)}]$$

- Use a reinforcement learning algorithm, like Proximal Policy Optimization (PPO) to maximize the above

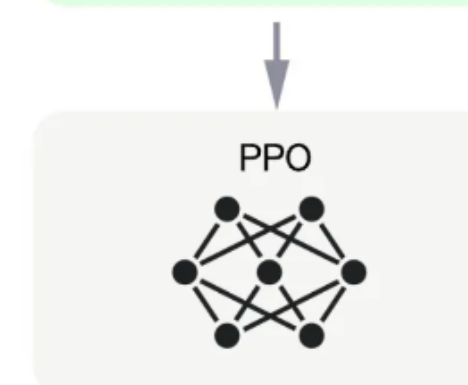
Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

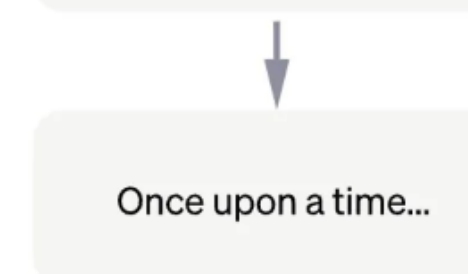
A new prompt is sampled from the dataset.



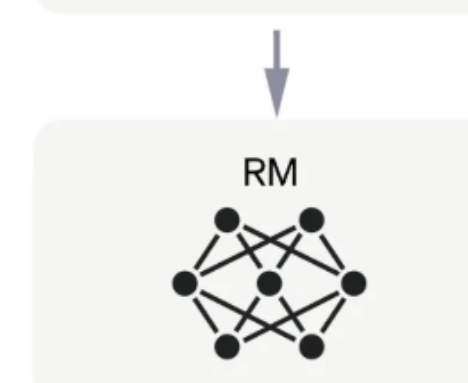
The PPO model is initialized from the supervised policy.



The policy generates an output.



The reward model calculates a reward for the output.



The reward is used to update the policy using PPO.



Reinforcement Learning

Step 3

Optimize a policy against the reward model using the PPO reinforcement learning algorithm.

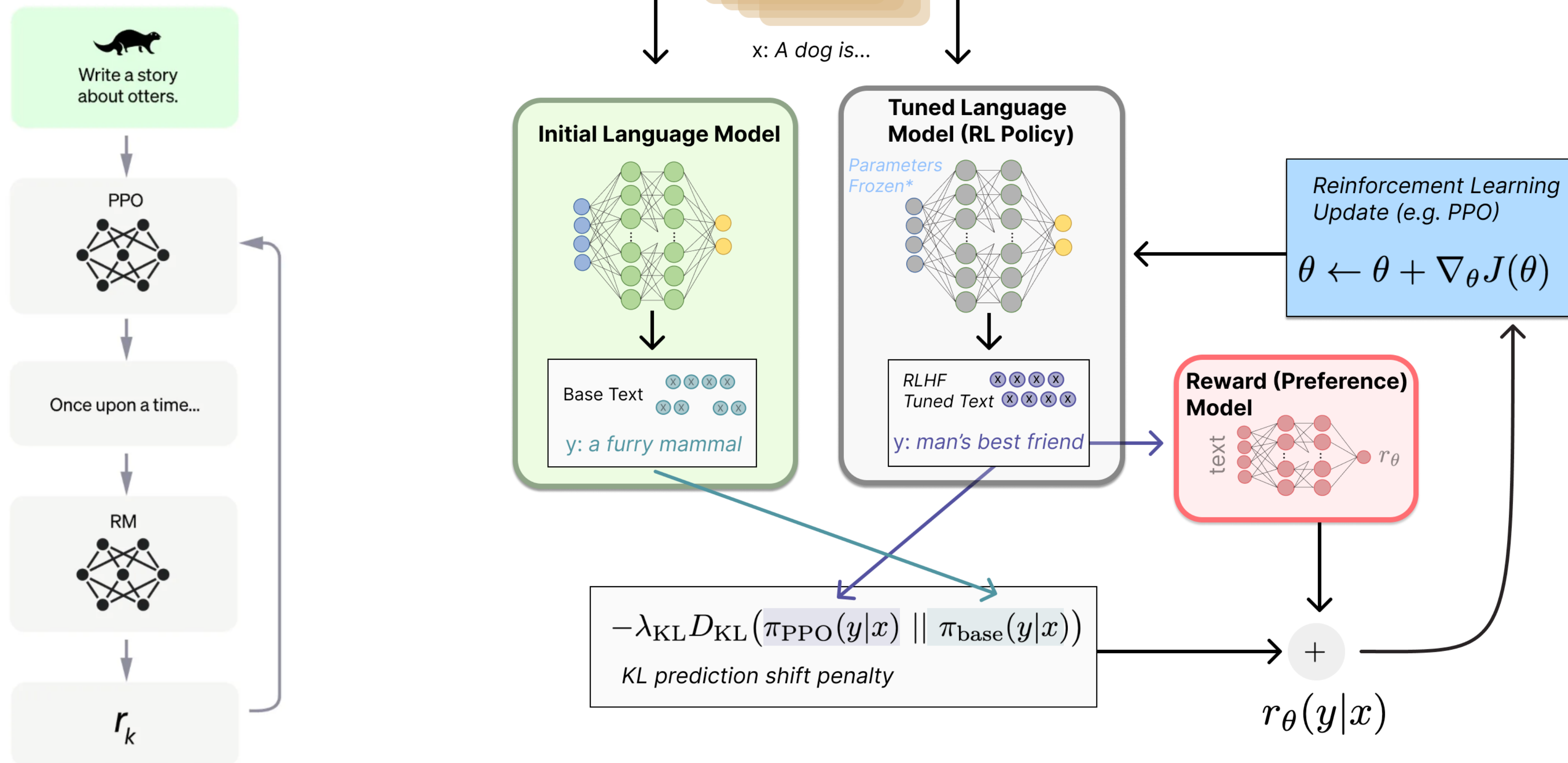
A new prompt is sampled from the dataset.

The PPO model is initialized from the supervised policy.

The policy generates an output.

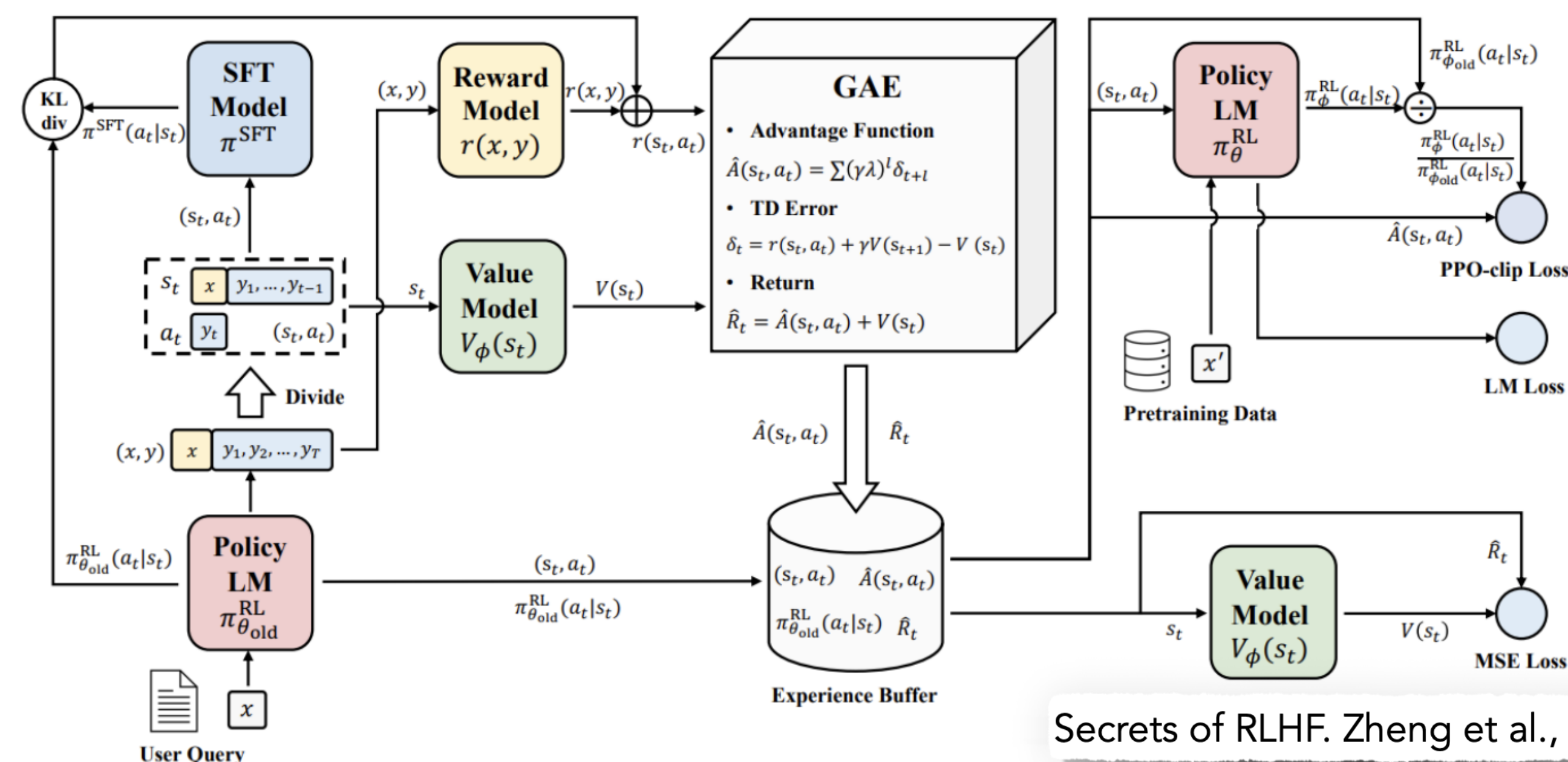
The reward model calculates a reward for the output.

The reward is used to update the policy using PPO.



RLHF to DPO

- Reinforcement Learning is tricky to train well, as well as computationally expensive
- Can we do supervised learning instead?
- Direct Preference Optimization (DPO)
[Rafailov et al., 2023]!



- Clever trick: we really only need the difference between the rewards for preferred output (y_w) and dispreferred output (y_l)
- Change the reward model $RM_{\theta}(x, y)$ as a modification of the language model itself: $p_{\theta}^{RL}(\hat{y}|x)$
- Everything is now a supervised learning objective!

$$L_{DPO}(\theta) = -\mathbb{E}_{(x, y_l, y_w) \sim D} \left[\log \sigma \left(\beta \log \frac{p_{\theta}^{RL}(y_w|x)}{p^{SFT}(y_w|x)} - \beta \log \frac{p_{\theta}^{RL}(y_l|x)}{p^{SFT}(y_l|x)} \right) \right]$$

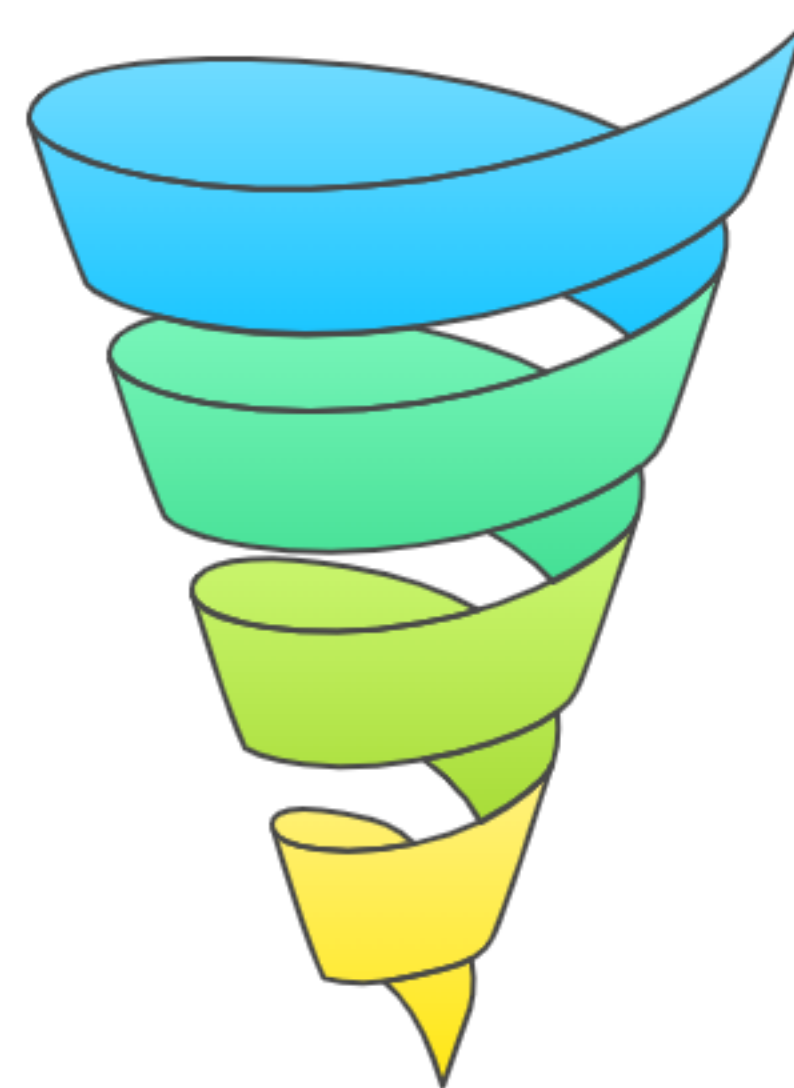
RLHF for powerful reasoning models

- Mathematical reasoning is now possible in language models due to powerful RLHF techniques

- Demonstrated by Deepseek-R1 and other

DeepSeek R1's Reasoning Process

- Problem Analysis**
The model analyzes the query to understand its components.
- Step-by-Step Breakdown**
The model breaks the problem into manageable steps.
- Insight Generation**
The model generates insights based on the analysis.
- Backtracking**
The model revisits steps to correct or refine the process.



chat.deepseek.com
DeepSeek - Into the Unknown

deepseek

If a train leaves Chicago traveling West at 70 miles per hour, and a second train leaves San Francisco traveling East at 80 miles per hour, when will the two trains meet?

Thought for 21 seconds ^

Alright, so I've got this problem here: there's a train leaving Chicago going west at 70 miles per hour, and another train leaving San Francisco going east at 80 miles per hour. I need to figure out when they'll meet. Okay, let's see.

First off, I need to understand the setup. Chicago is in the east, and San Francisco is in the west, right? So, the two cities are on opposite ends of the country. The trains are moving towards each other, one going west and the other east. That means they're heading straight toward each other.

Now, I think I need to know the distance between Chicago and San Francisco to solve this. Do I know that off the top of my head? I'm not sure, but I think it's somewhere around 2000 miles. Let me double-check that. Yeah, a quick search says it's approximately 1,800 miles apart. Okay, so I'll go with 1,800 miles for this problem.

So, both trains are moving toward each other, one at 70 mph and the other at 80 mph. I guess I need to find out how long it takes for the sum of the distances they travel to equal the total distance between the two cities.

Wait a minute, if one is going west and the other east, and they're moving toward each other, their speeds should add up because they're approaching each other. So, the combined speed

Message DeepSeek

Deep Think (42 messages left today) NEW

AI-generated, for reference only

chatgpt.com
Train Meeting Time

ChatGPT o1-preview

If a train leaves Chicago traveling West at 70 miles per hour, and a second train leaves San Francisco traveling East at 80 miles per hour, when will the two trains meet?

Thought for 5 seconds

To determine when the two trains will meet, we need to calculate the time it takes for them to cover the distance between Chicago and San Francisco while traveling towards each other.

1. Determine the Distance Between Chicago and San Francisco:

The approximate railway distance between Chicago and San Francisco is about **2,438 miles**. This is based on the route of the California Zephyr, an Amtrak train that runs between these two cities.

2. Calculate the Combined Speed:

Since the trains are moving towards each other, their speeds add up.

- Speed of Train A (from Chicago): **70 mph**
- Speed of Train B (from San Francisco): **80 mph**

Combined Speed = 70 mph + 80 mph = 150 mph

3. Calculate the Time Until They Meet:

Use the formula:

$$\text{Time} = \frac{\text{Distance}}{\text{Speed}}$$

Message ChatGPT

Tiernan Ray

ChatGPT can make mistakes. Check important info.

Preference Tuning: Parting Thoughts

- We want to optimize for human preferences as it's an important step towards LLM safety
 - Instead of humans writing the answers or giving uncalibrated scores, we get humans to rank different LM generated answers
- Reinforcement learning from human feedback
 - Train an explicit reward model on comparison data to predict a score for a given completion
 - Optimize the LM to maximize the predicted score without deviating too much
 - Very effective when tuned well, computationally expensive and tricky to get right
- Direct Preference Optimization
 - Optimize LM parameters directly on preference data
 - Simple and effective, similar properties and performance to RLHF

Three Stages of Training LLMs

Stage 1

- Pre-training on large corpus of text
- Produces a Base Language Model
- Continued Pre-training for domain adaptation (optional; sometimes called mid-training, stage 1.5)

Stage 2

- Post-training for Task Adaptation
- Seq2Seq Instruction Tuning (Supervised Finetuning, different meaning than BERT-style classification tasks)

Stage 3

- Post-training for Preference Alignment
 - Either Reinforcement Learning with Human Feedback : Train a supervised classifier (reward model) on human demonstrations to provide feedback to LM. Reinforcement learning to maximize rewards given by reward model
 - Or, train with a preferred and a dispreferred generation (more popular now)

Inference

Prompting with Instructions and Demonstrations (also called examples, shots)

Large Language Models

What are Large Language Models?

- Models with many, many, parameters
 - Deeper layers. The largest T5 model had 11B parameters. GPT-3 has 175B parameters. Llama-3 has a 405B parameter variant.
 - More context. GPT-2 has a context length of 1024 tokens, GPT-4 and Llama-2 have 8192, Llama-3 has 128K!
 - LLMs are generally trained by filling the full context window with text. If documents are shorter than this, multiple documents are packed into the window with a special end-of-text token between them.
- Models trained on many, many tokens
 - Training data size is measured in # tokens
 - Batch size of the largest GPT-3 model is 3.2M tokens

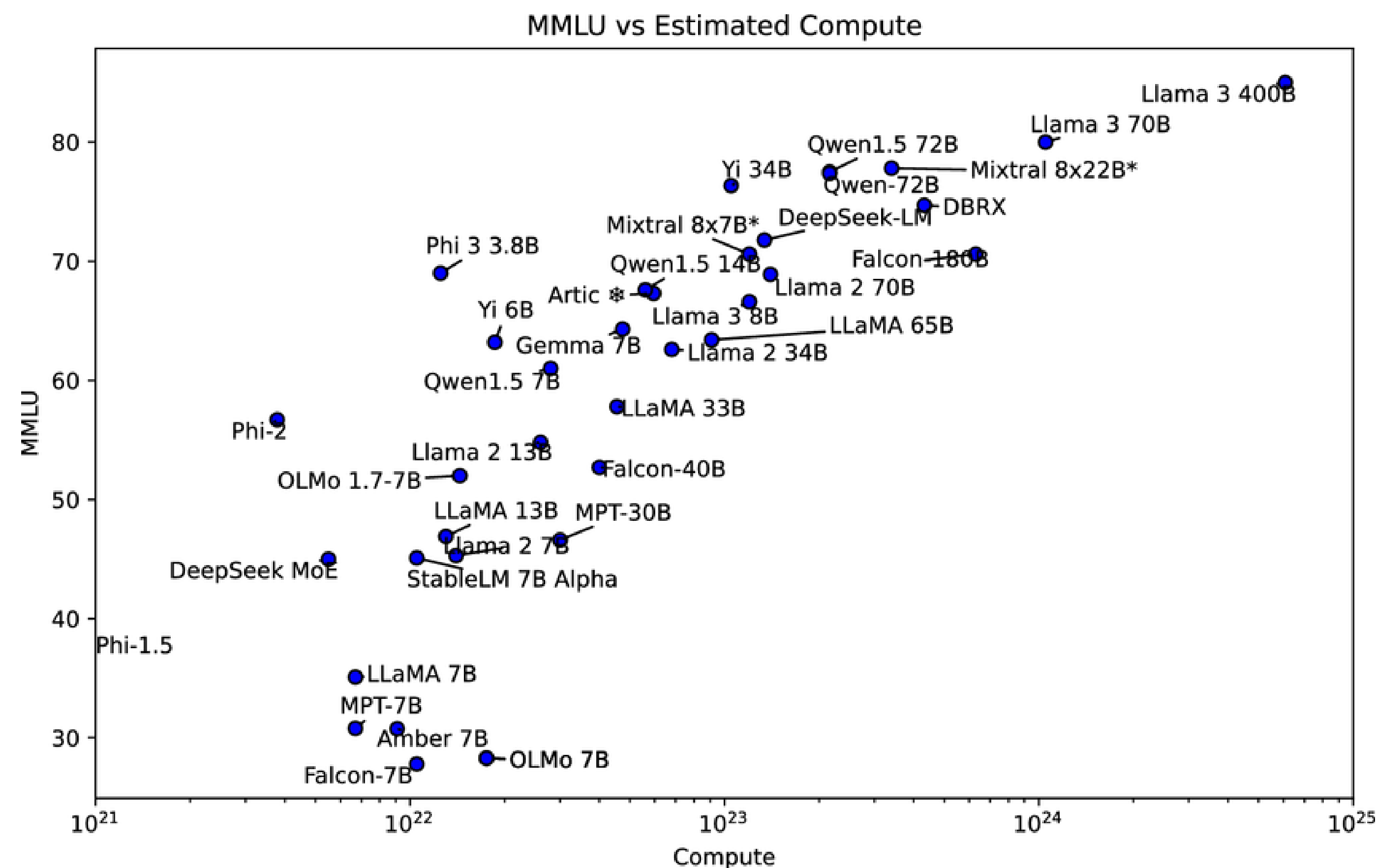
OpenAI model's version	GPT-3 (ada, babbage, curie, davinci)	GPT-3.5 (gpt-3.5-turbo, gpt-3.5-turbo-0301, text-davinci-003, text-davinci-002)*	GPT-4-8K
Context length (max request)	2,049	4,096	8,192
Number of English words	~1,500	~3,000	~6,000
Number of single-spaced pages of English text	3	6	12

Source: [Neoteric](#)

Model	Size (# Parameters)	Training Tokens
LaMDA (Thoppilan et al., 2022)	137 Billion	168 Billion
GPT-3 (Brown et al., 2020)	175 Billion	300 Billion
Jurassic (Lieber et al., 2021)	178 Billion	300 Billion
Gopher (Rae et al., 2021)	280 Billion	300 Billion
MT-NLG 530B (Smith et al., 2022)	530 Billion	270 Billion
<i>Chinchilla</i>	70 Billion	1.4 Trillion

The Effect of LLM Scale

- Models designed with high capacity/scale can potentially achieve high performance
- Main factors determining model scale:
 - Model architecture (Transformer vs. RNN vs. State Space Model)
 - Training FLOPs
 - # model parameters
 - # tokens of pretraining data



The Scaling “Laws” of LLMs

- Predict loss given parameter size, data size, computation
 - Learn a relationship from a small number of samples
 - Estimate the model behaviors at high-capacity by experimenting on low-capacity models

Kaplan et al., 2020 <https://arxiv.org/abs/2001.08361>

One example:

Chinchilla Scaling Law Formulation [1]

$$\hat{L}(N, D) \triangleq E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}.$$

L: Loss

N: # Model Parameter

D: # Training Tokens

E: A constant capturing the Entropy of the text

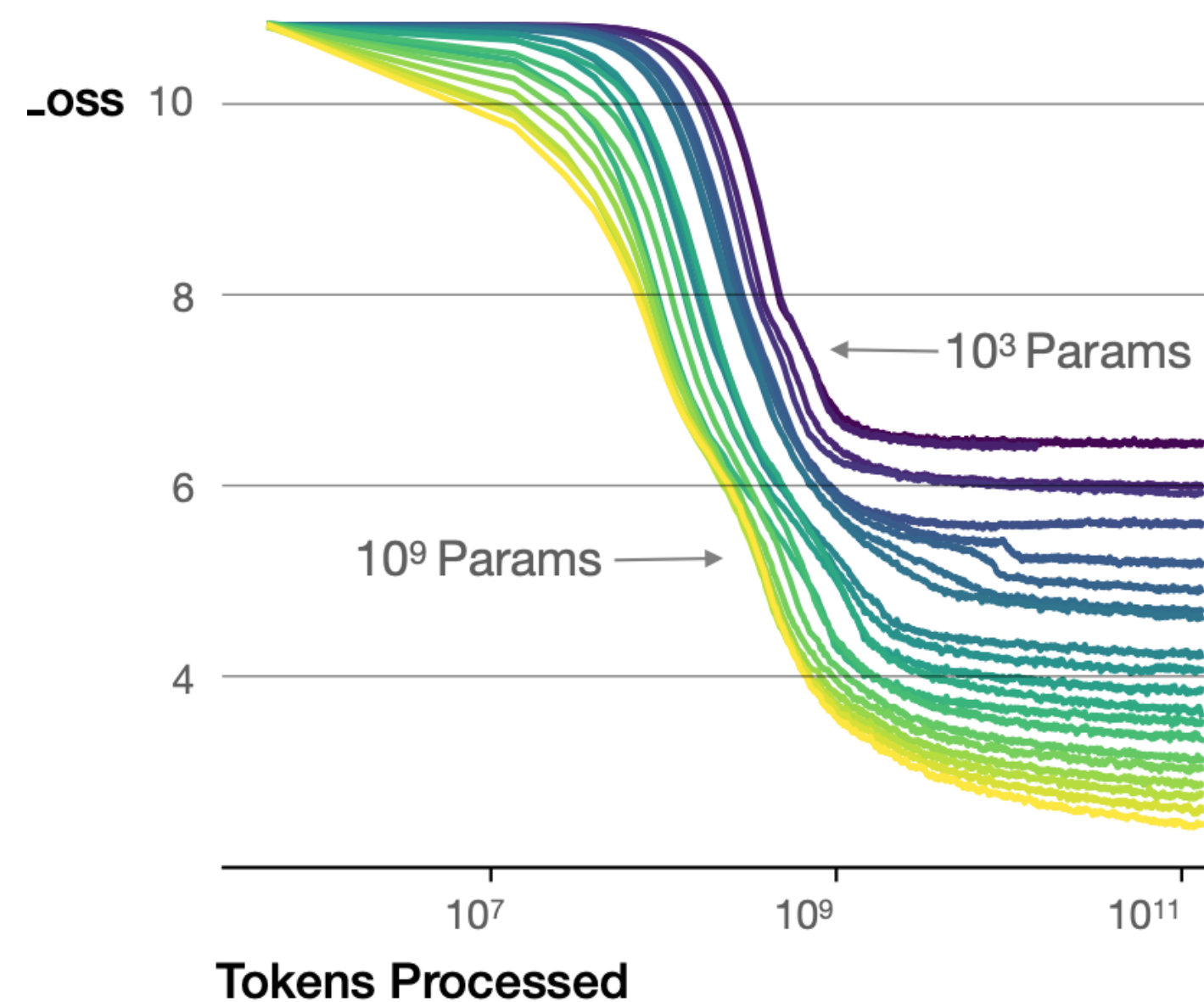
E, A, B, α , and β are to be fit during experiments

The loss of an LLM scales as a power-law with each of these properties of model training

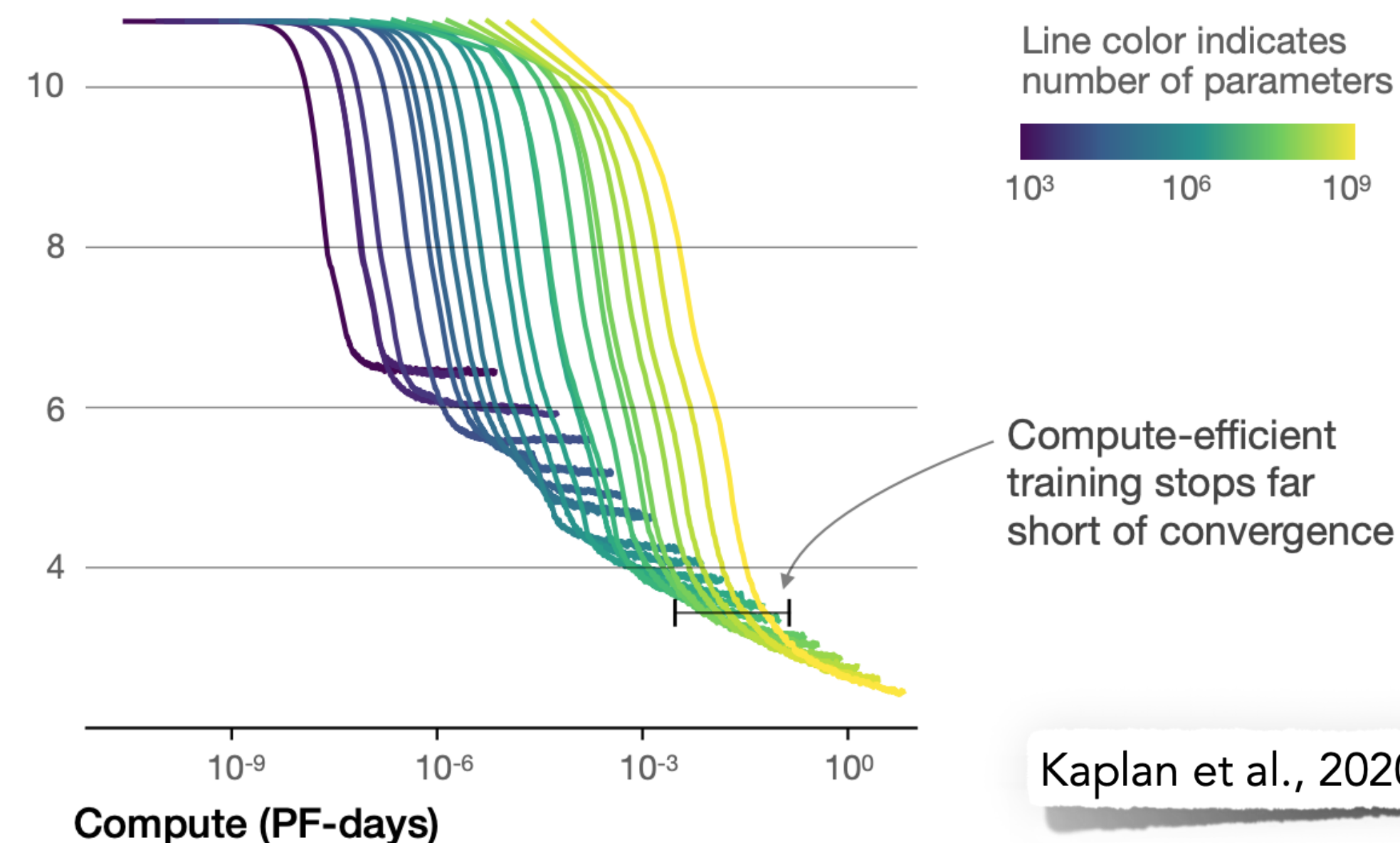
The Scaling “Laws” of LLMs

- Other architectural details such as network width or depth have minimal effects within a wide range
- Determines the optimal allocation of a fixed compute budget
 - For instance, it can help us train very large models on a relatively modest amount of data and stop significantly before convergence
 - Or smaller models on larger data (e.g. Chinchilla LM)

Larger models require **fewer samples** to reach the same performance



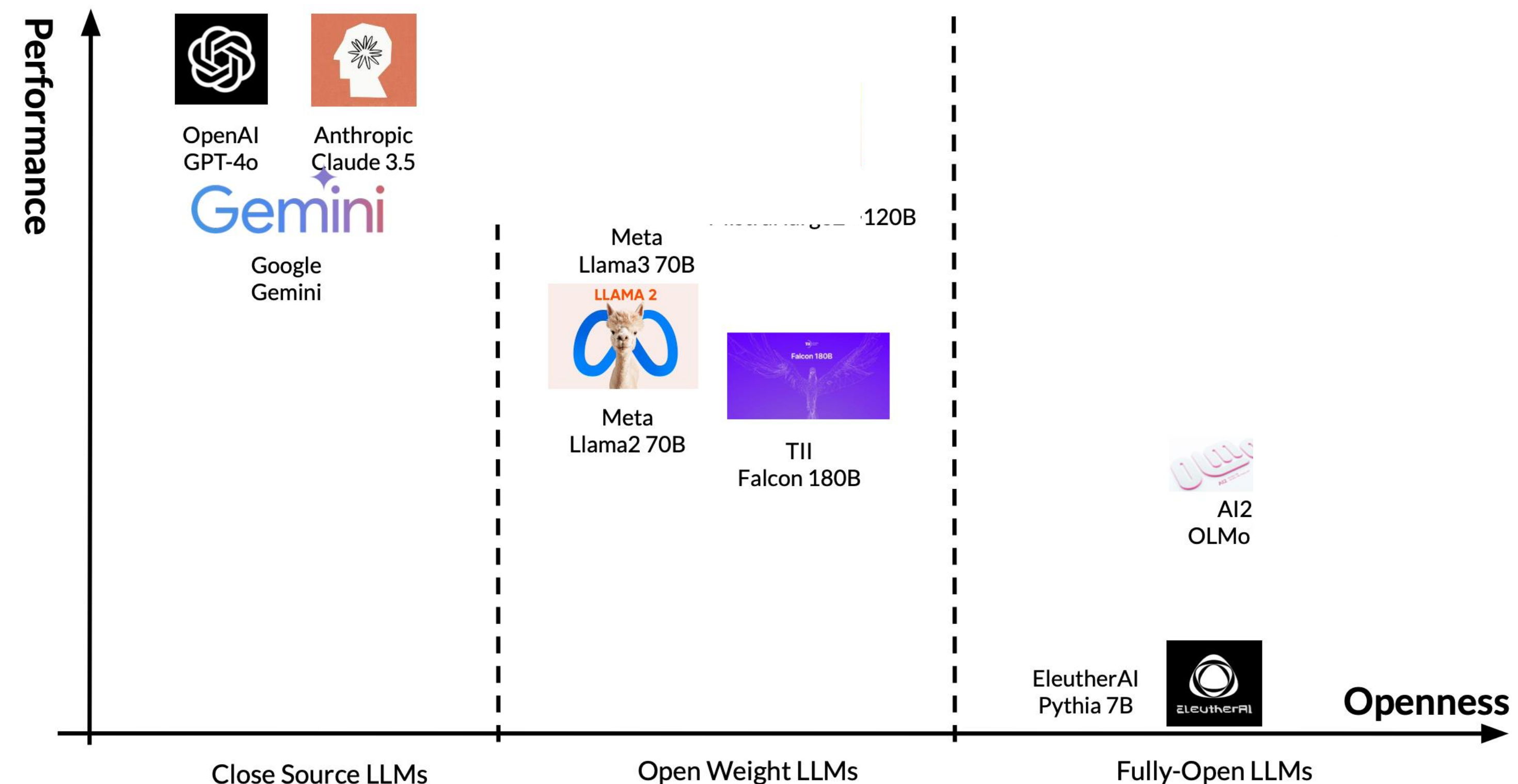
The optimal model size grows smoothly with the loss target and compute budget



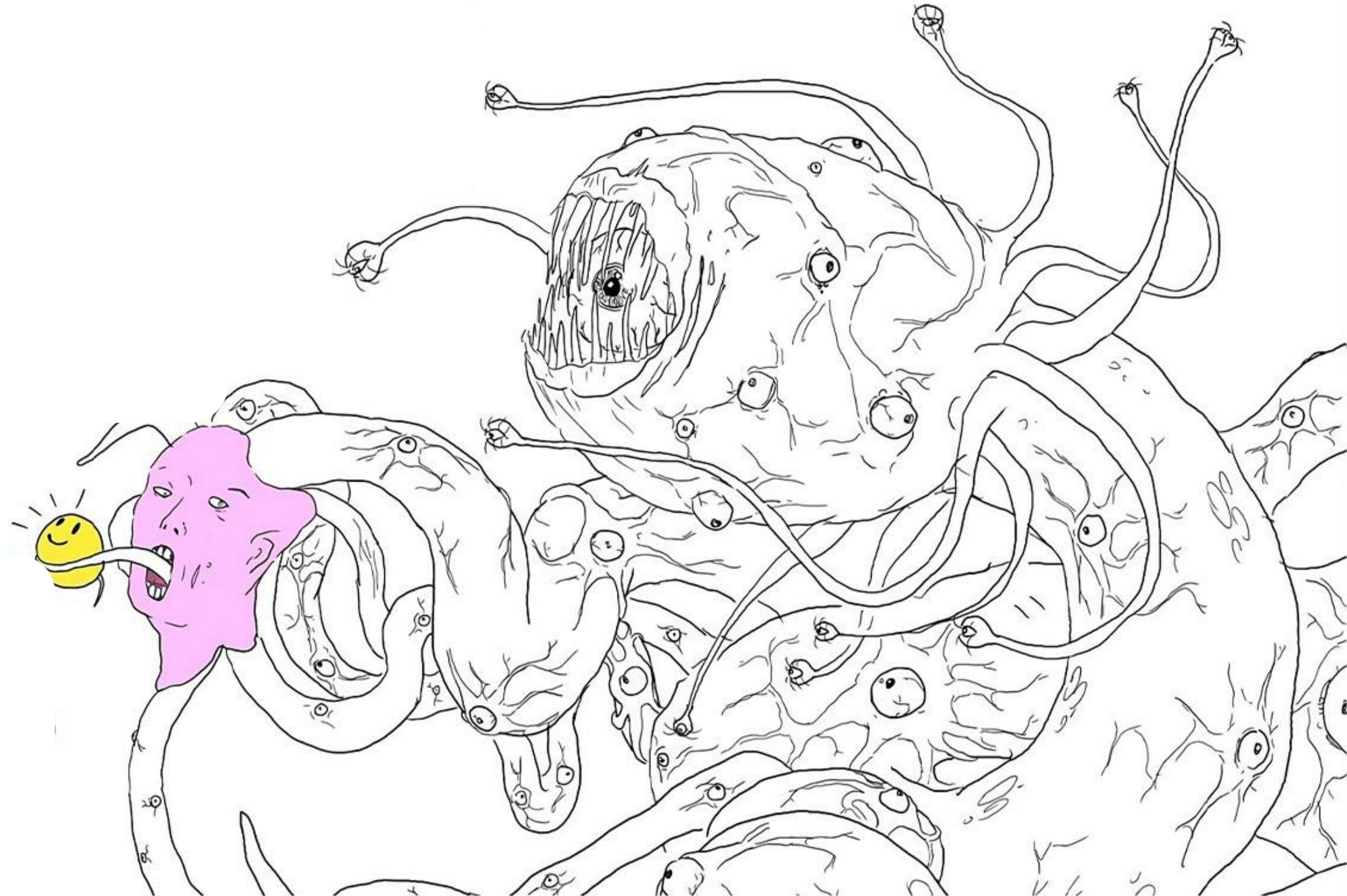
Kaplan et al., 2020 <https://arxiv.org/abs/2001.08361>

Open-Source, Open-Weight vs. Closed LLMs

- Open-Source: everything is released. Can completely reproduce, provided you have the compute
 - e.g. Allen AI's OLMo, Eleuther AI's Pythia
 - Levels the playing field for LLM development via knowledge sharing
- Open Weight: only the final model weights are released, but no data, code etc.
 - e.g. DeepSeek-R1, Llama-3, Qwen, Mixtral
- Closed-Source / API-protected: Most popular and also, the best performing
 - e.g. GPT-4o, Claude-4.5



Training LLMs



Shoggoth; slide credit: Justin Cho